



UNIVERSITATEA DIN CRAIOVA
FACULTATEA DE AUTOMATICĂ, CALCULATOARE ȘI
ELECTRONICĂ
DEPARTAMENTUL DE AUTOMATICĂ ȘI ELECTRONICĂ



PROIECT DE DIPLOMĂ

Anca Maria Băcănoiu

COORDONATOR ȘTIINȚIFIC

Asist. drd. ing. Dragoș Cîrciumariu

Prof. univ. dr. ing. Dorin Sendrescu

CRAIOVA, ROMÂNIA

Iulie 2026



UNIVERSITATEA DIN CRAIOVA
FACULTATEA DE AUTOMATICĂ, CALCULATOARE ȘI
ELECTRONICĂ
DEPARTAMENTUL DE AUTOMATICĂ ȘI ELECTRONICĂ



SISTEM IoT SMART HOME

Anca Maria Băcănoiu

COORDONATOR ȘTIINȚIFIC

Asist. drd. ing. Dragoș Cîrciumariu

Prof. univ. dr. ing. Dorin Sendrescu

CRAIOVA, ROMÂNIA

Iulie 2026

„Învățătura este o comoară care își urmează stăpânul pretutindeni.”

Proverb popular

DECLARAȚIA DE ORIGINALITATE

Subsemnatul Anca Maria Băcănoiu, student la specializarea Automatică și Informatică Aplicată din cadrul Facultății de Automatică, Calculatoare și Electronică a Universității din Craiova, certific prin prezenta că am luat la cunoștință de cele prezentate mai jos și că îmi asum, în acest context originalitatea proiectului meu de licență:

- cu titlul Sistem IoT Smart Home,
- coordonată de Asist. drd. ing. Dragoș Cîrciumariu și Prof. univ. dr. ing. Dorin Sendrescu,
- prezentată în sesiunea Iulie 2026.

La elaborarea proiectului de licență, se consideră plagiat una dintre următoarele acțiuni:

1. reproducerea exactă a cuvintelor unui alt autor, dintr-o altă lucrare, în limba română sau prin traducere dintr-o altă limbă, dacă se omit ghilimele și referința precisă,
2. redarea cu alte cuvinte, reformularea prin cuvinte proprii sau rezumarea ideilor din alte lucrări, dacă nu se indică sursa bibliografică,
3. prezentarea unor date experimentale obținute sau a unor aplicații realizate de alți autori fără menționarea corectă a acestor surse,
4. însușirea totală sau parțială a unei lucrări în care regulile de mai sus sunt respectate, dar care are alt autor.

Pentru evitarea acestor situații neplăcute se recomandă:

- plasarea între ghilimele a citatelor directe și indicarea referinței într-o listă corespunzătoare la sfârșitul lucrării,
- indicarea în text a reformulării unei idei, opinii sau teorii și corespunzător în lista de referințe a sursei originale de la care s-a făcut preluarea,
- precizarea sursei de la care s-au preluat date experimentale, descrieri tehnice, figuri, imagini, statistici, tabele et caetera,
- precizarea referințelor poate fi omisă dacă se folosesc informații sau teorii arhicunoscute, a căror paternitate este unanim cunoscută și acceptată.

Data,

Semnătura candidatului,



UNIVERSITATEA DIN CRAIOVA
Facultatea de Automatică, Calculatoare și Electronică

Departamentul de Automatică și Electronică

Aprobat la data de
Șef de departament,
Prof. dr. ing.
Cătălin
CONSTANTINESCU

PROIECTUL DE DIPLOMĂ

Numele și prenumele studentului/-ei:	Anca Maria BĂCĂNOIU
Enunțul temei:	Sistem IoT Smart Home
Datele de pornire:	Platforma hardware ESP32-WROOM-32, senzori de mediu (BMP280, MQ135, TEMT6000, BISS0001), cititor RFID RC522, afișaj LCD HD44780 1602, broker MQTT HiveMQ Cloud, mediul de dezvoltare Arduino IDE 2.x cu ESP32 Arduino Core v3.x.
Conținutul proiectului:	Introducere, Capitolul 1. Fundamente teoretice și tehnologii utilizate, Capitolul 2. Analiza cerințelor și proiectarea funcțională, Capitolul 3. Arhitectura hardware a sistemului, Capitolul 4. Implementarea firmware-ului ESP32, Capitolul 5. Implementarea aplicației PWA și comunicația MQTT, Concluzii
Material grafic obligatoriu:	diagrame de stare, imagini din aplicație, tabele statice
Consultații:	Periodice
Conducătorul științific (titlul, nume și prenume, semnătura):	Asist. drd. ing. Dragoș Cîrciumariu Prof. univ. dr. ing. Dorin Sendrescu
Data eliberării temei:	
Termenul estimat de predare a proiectului:	16.06.2026
Data predării proiectului de către student și semnătura acestuia:	19.06.2026



UNIVERSITATEA DIN CRAIOVA
Facultatea de Automatică, Calculatoare și Electronică

Departamentul de Automatică și Electronică

REFERATUL CONDUCĂTORULUI ȘTIINȚIFIC

Numele și prenumele candidatului/-ei: Băcănioiu Anca Maria
Specializarea: Automatică și Informatică Aplicată
Titlul proiectului: Sistem IoT Smart Home

Locația în care s-a realizat practica de documentare (se bifează una sau mai multe din opțiunile din dreapta):
În facultate ☐
În producție ☐
În cercetare ☐
Altă locație:

În urma analizei lucrării candidatului au fost constatate următoarele:

Nivelul documentării		Insuficient ☐	Satisfăcător ☐	Bine ☐	Foarte bine ☐
Tipul proiectului		Cercetare ☐	Proiectare ☐	Realizare practică ☐	Altul [se detaliază]
Aparatul matematic utilizat		Simplu ☐	Mediu ☐	Complex ☐	Absent ☐
Utilitate		Contract de cercetare ☐	Cercetare internă ☐	Utilare ☐	Altul [se detaliază]
Redactarea lucrării		Insuficient ☐	Satisfăcător ☐	Bine ☐	Foarte bine ☐
Partea grafică, desene		Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
Realizare a practică	Contribuția autorului	Insuficientă ☐	Satisfăcătoare ☐	Mare ☐	Foarte mare ☐
	Complexitatea temei	Simplă ☐	Medie ☐	Mare ☐	Complexă ☐
	Analiza cerințelor	Insuficient ☐	Satisfăcător ☐	Bine ☐	Foarte bine ☐
	Arhitectura	Simplă ☐	Medie ☐	Mare ☐	Complexă ☐

	Întocmirea specificațiilor funcționale	Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
	Implementarea	Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
	Testarea	Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
	Funcționarea	Da ☐	Parțială ☐	Nu ☐	
Rezultate experimentale		Experiment propriu ☐		Preluare din bibliografie ☐	
Bibliografie		Cărți	Reviste	Articole	Referințe web
Comentarii și observații					

În concluzie, se propune:

ADMITEREA PROIECTULUI ☐	RESPINGEREA PROIECTULUI ☐
--	--

Data,

Semnătura conducătorului științific,

REZUMATUL PROIECTULUI

Lucrarea de față abordează proiectarea și implementarea unui sistem de automatizare rezidențială cu funcții de monitorizare a mediului ambiant, control al accesului și gestionare a securității, avându-l la bază pe microcontrollerul ESP32-WROOM-32 și pe protocolul de comunicație MQTT. Sistemul a fost conceput ca un prototip funcțional complet, capabil să demonstreze integrarea coerentă a mai multor tehnologii din domenii diferite: electronică analogică și digitală, programare de sisteme embedded, protocoale de comunicație IoT și dezvoltare web.

Ansamblul hardware integrat combină cinci tipuri de senzori cu funcții distincte: modulul BMP280 pentru achiziția temperaturii și presiunii atmosferice prin interfața I2C, fototranzistorul TEMA6000 pentru cuantificarea luminozității ambientale prin convertor analog-digital, senzorul semiconductor MQ135 destinat evaluării calității aerului prin aceeași cale de conversie analogică, modulul pasiv infraroșu BISS0001 cu circuit de prelucrare integrat pentru detecția prezenței persoanelor în zona monitorizată și cititorul RFID RC522 pentru identificarea utilizatorilor prin carduri Mifare la frecvența de 13,56 MHz. Actuatorii sistemului includ un LED RGB cu control independent al intensității pe fiecare din cele trei canale de culoare prin modulație în lățime de impuls, un buzzer activ gestionat în mod non-blocant și un afișaj LCD tip HD44780 de 16 caractere pe 2 rânduri.

Logica de control este implementată ca firmware rulând pe microcontrollerul ESP32, conectat la internet prin WiFi 802.11 b/g/n integrat. Comunicarea bidirecțională cu aplicația de monitorizare se desfășoară prin protocolul MQTT, cu brokerul HiveMQ Cloud ca intermediar în cloud. Datele achiziționate periodic de la senzori sunt publicate pe topicuri MQTT dedicate, în format JSON, iar comenzile de configurare și control sunt recepționate pe canale separate. Această arhitectură de tip publish/subscribe separă complet producătorii de consumatorii de mesaje, permițând adăugarea ulterioară de noi clienți fără modificarea firmware-ului.

Sistemul de securitate implementat funcționează pe baza unei mașini de stări cu patru stări distincte, incluzând o fază de alarmă silențioasă de 60 de secunde la detectarea mișcării, timp în care utilizatorul autorizat poate dezarma sistemul prin prezentarea unui card RFID valabil. Depășirea pragului de concentrație a gazelor detectat de MQ135 declanșează o alarmă cu prioritate maximă, independentă de starea de armare. Configurațiile asociate fiecărui card RFID — culoarea LED-ului, comportamentul la zi/noapte, semnalul sonor la citire — sunt stocate persistent în memoria non-volatilă a microcontrollerului.

Interfața de utilizare constă dintr-o aplicație web de tip Progressive Web App, implementată într-un singur fișier HTML fără dependențe de framework, care se conectează direct la brokerul

MQTT prin WebSocket securizat și oferă acces complet la funcționalitățile sistemului de pe orice dispozitiv cu browser modern.

Termenii cheie: ESP32, BMP180, RC-522, MQTT, PIR, TEMT6000, Smart Home, LED, RFID, PWA, Broker MQTT, Web Socket, IoT.

MULȚUMIRI

Doresc să adresez mulțumiri coordonatorului științific, Asist. drd. ing. Dragoș Cîrciumariu și Prof. univ. dr. ing. Dorin Sendrescu, pentru îndrumarea acordată pe parcursul elaborării acestei lucrări, pentru disponibilitatea manifestată constant în clarificarea aspectelor tehnice și pentru observațiile critice care au contribuit semnificativ la îmbunătățirea calității conținutului.

Mulțumesc familiei mele pentru susținerea necondiționată și răbdarea acordate pe întreaga perioadă a studiilor universitare, fără de care această lucrare nu ar fi putut fi finalizată.

Adresez, de asemenea, mulțumiri colegilor și cadrelor didactice din cadrul Facultății de Automatică, Calculatoare și Electronică a Universității din Craiova, al căror aport în formarea mea profesională s-a reflectat direct în modul de abordare a temei prezentei lucrări.

CUPRINSUL

INTRODUCERE.....	1
Capitolul 1. Fundamente teoretice și tehnologii utilizate.....	5
1.1. Conceptul de Smart Home și rolul IoT.....	5
1.2. Microcontrolere în sisteme Smart Home. Rolul ESP32.....	6
1.3. Protocolul MQTT în aplicații IoT.....	8
1.4. Aplicațiile PWA pentru controlul sistemelor IoT.....	9
1.5. Brokerul cloud HiveMQ și comunicația prin WebSocket.....	10
1.6. Considerații privind securitatea.....	11
Capitolul 2. Analiza cerințelor și proiectarea funcțională.....	13
2.1. Descrierea studiului de caz.....	13
2.2. Actorii sistemului.....	13
2.3. Cazuri de utilizare.....	15
2.4. Contractul de comunicație MQTT.....	18
Capitolul 3. Arhitectura hardware a sistemului.....	22
3.1. Vedere de ansamblu asupra sistemului hardware.....	22
3.2. Modulul ESP32.....	23
3.3. Senzorii utilizați.....	24
3.4. Actuatori și elemente de semnalizare.....	25
3.5. Modulul RFID RC522.....	27
3.6. Schema de conectare și pinout.....	29
Capitolul 4. Implementarea firmware-ului ESP32.....	32
4.1. Mediul de dezvoltare și bibliotecile utilizate.....	32
4.2. Inițializarea sistemului.....	34
4.3. Bucla principală de execuție.....	36
4.4. Citirea senzorilor și publicarea datelor.....	39
4.5. Comunicarea MQTT în firmware.....	40
4.6. Logica de alarmă.....	44
4.7. Managementul RFID și stocarea cardurilor.....	47
4.8. Controlul LED RGB, LCD și buzzer.....	50
4.9. Jurnalizarea evenimentelor.....	52
Capitolul 5. Implementarea aplicației PWA și comunicația MQTT.....	54
5.1. Structura aplicației PWA.....	54
5.2. Conectarea la broker prin Paho MQTT.....	55
5.3. Dashboard-ul pentru monitorizare.....	57
5.4. Controlul LED RGB.....	58
5.5. Gestiunea RFID.....	59
5.6. Loguri, setări MQTT și persistență locală.....	60
Capitolul 6. Concluzii.....	62
Bibliografie.....	68
Referințe Web.....	69
A. CODUL SURSĂ.....	71
B. SITE-UL WEB AL PROIECTULUI.....	72

C. CD / DVD / STICK USB.....	73
-------------------------------------	-----------

Lista Figurilor

Figura 1.1 Modelul publish/subscribe în MQTT.....	9
Figura 2.1 Diagrama actorilor sistemului.....	16
Figura 2.2 Diagrama cazurilor de utilizare.....	17
Figura 3.1 Diagrama bloc hardware a sistemului Smart Home.....	23
Figura 3.2. Fluxul hardware pentru citirea și tratarea unui card RFID.....	29
Figura 4.1 Flowchart pentru inițializarea sistemului în funcția setup.....	38
Figura 4.2 Flowchart pentru bucla principală de execuție.....	41
Figura 4.3 Fluxul MQTT dintre ESP32, broker și aplicația PWA.....	45
Figura 4.4 Diagrama de stare a alarmei de gaz.....	48
Figura 4.5 Diagrama de stare a alarmei de mișcare.....	50
Figura 4.6 Diagrama de secvență pentru adăugarea unui card RFID.....	52
Figura 5.1. Structura ecranelor aplicației PWA.....	57
Figura 5.2. Secvența de conectare a aplicației PWA la brokerul HiveMQ.....	59
Figura 5.3. Ecranul de control al LED-ului RGB.....	62

Lista Tabelelor

Tabel I.1. Obiectivele lucrării și modul de realizare.....	3
Tabelul 3.1. Actuatori și elemente de afișare utilizate în sistem.....	27
Tabelul 3.2. Alocarea pinilor ESP32 în sistemul Smart Home (1/3).....	30
Tabelul 4.1. Comenzi MQTT tratate de ESP32 (1/2).....	44

INTRODUCERE

Lucrarea pornește de la un scenariu practic: realizarea unui prototip care să poată supraveghea anumite condiții dintr-un spațiu interior și să permită utilizatorului să trimită comenzi către dispozitiv printr-o aplicație web. În acest sens, sistemul nu se limitează la citirea unei singure valori sau la controlul unui singur actuator, ci reunește mai multe funcții specifice unei locuințe inteligente: monitorizarea parametrilor de mediu, semnalizarea locală, detectarea mișcării, identificarea prin card RFID și comunicarea la distanță printr-un broker MQTT.

Proiectul propus și dezvoltat în această lucrare este un sistem Smart Home cu arhitectură IoT, construit în jurul microcontrolerului ESP32, al unei aplicații PWA și al comunicației MQTT prin broker cloud. În forma implementată, sistemul permite citirea temperaturii, presiunii, nivelului de lumină, valorii unui senzor de gaz și stării unui senzor PIR. Pe lângă monitorizare, utilizatorul poate controla un LED RGB, poate arma sau dezarma alarma, poate gestiona carduri RFID și poate consulta mesajele de stare afișate în aplicație.

Legătura dintre dispozitivul fizic și aplicația PWA este realizată prin MQTT. ESP32 publică valorile senzorilor și evenimentele importante pe topicuri dedicate, iar aplicația se abonează la aceste topicuri pentru a actualiza interfața. În sens invers, aplicația publică mesaje de comandă, pe care firmware-ul le interpretează pentru controlul LED-ului, al alarmei sau al funcțiilor RFID. Astfel, brokerul MQTT funcționează ca intermediar între partea hardware și interfața software, fără ca aplicația să comunice direct cu pinii sau senzorii conectați la ESP32.

Alegerea plăcii ESP32 a fost determinată de nevoia unui microcontroler care să poată gestiona atât componente hardware locale, cât și comunicația wireless. În proiectul realizat, ESP32 funcționează ca unitate centrală a sistemului: citește valorile senzorilor, actualizează stările interne, controlează LED-ul RGB, buzzerul și LCD-ul, gestionează modulul RFID și transmite datele către brokerul MQTT. Platforma este potrivită pentru această arhitectură deoarece pune la dispoziție conectivitate WiFi integrată, pini digitali, intrări analogice și interfețe de comunicație precum I2C și SPI. Astfel, componente diferite pot fi conectate la aceeași placă fără a fi necesar un calculator extern sau un modul separat de comunicație.

MQTT a fost ales deoarece se potrivește modului în care sistemul schimbă date între ESP32 și aplicația PWA. Microcontrolerul publică valorile senzorilor și evenimentele importante pe topicuri dedicate, iar aplicația se abonează la aceste topicuri pentru a actualiza dashboard-ul. În sens invers, aplicația publică mesaje de comandă pentru LED, alarmă, RFID și loguri, iar ESP32 le primește și le interpretează în firmware. Această structură reduce dependența dintre aplicație și dispozitivul fizic, deoarece brokerul MQTT devine punctul comun prin care cele două componente comunică. În plus, aceeași organizare permite extinderea sistemului cu alte aplicații, senzori sau dispozitive care pot folosi aceleași topicuri.

Pentru brokerul MQTT a fost utilizată platforma HiveMQ Cloud, în varianta gratuită, deoarece oferă o soluție accesibilă pentru testarea unui prototip IoT fără configurarea unui server propriu. În cazul aplicației PWA, conexiunea la broker se realizează prin WebSocket securizat, deoarece browserul nu folosește în mod direct conexiuni MQTT TCP obișnuite. Din acest motiv, aplicația utilizează clientul Eclipse Paho JavaScript, care permite conectarea la broker și schimbul de mesaje MQTT din browser. Această alegere a simplificat integrarea dintre aplicația web și sistemul fizic controlat de ESP32.

Aplicația PWA a fost aleasă pentru a oferi o interfață ușor de accesat de pe mai multe dispozitive. Utilizatorul poate deschide aplicația din browser, fără instalarea unei aplicații native dintr-un magazin de aplicații, iar interfața poate fi folosită de pe telefon, tabletă sau laptop. În proiect, aplicația include ecrane pentru monitorizarea senzorilor, controlul LED-ului RGB, gestiunea cardurilor RFID, configurarea cardurilor și consultarea jurnalului de evenimente. Prin această structură, aplicația devine panoul de control al sistemului, iar utilizatorul poate observa starea prototipului și poate trimite comenzi către ESP32 într-un mod rapid.

Motivația alegerii acestei teme pornește de la interesul tot mai mare pentru sisteme care permit monitorizarea și controlul locuinței dintr-o interfață unificată. În utilizarea de zi cu zi, multe dispozitive casnice ajung să fie controlate separat, prin aplicații diferite sau prin soluții care nu comunică între ele. Din acest motiv, un sistem Smart Home devine util atunci când poate reuni mai multe funcții într-o singură interfață și când poate fi adaptat ulterior fără schimbarea completă a arhitecturii.

Multe soluții existente pe piață oferă funcții avansate, însă sunt construite în ecosisteme închise, în care extinderea sau integrarea cu alte componente poate fi dificilă. Într-un proiect academic, o variantă mai potrivită este folosirea unor tehnologii accesibile și flexibile, care permit înțelegerea modului în care sunt conectate componentele hardware și software. Din acest motiv, lucrarea propune un prototip bazat pe ESP32, MQTT și aplicație PWA, în care fiecare parte a sistemului poate fi analizată, testată și modificată.

Scopul principal al lucrării este proiectarea și implementarea unui sistem Smart Home bazat pe ESP32, care comunică prin MQTT cu o aplicație PWA. Sistemul monitorizează temperatura, presiunea, nivelul de lumină, valoarea unui senzor de gaz și detecția de mișcare. Pe lângă aceste funcții, prototipul permite controlul unui LED RGB, gestionarea cardurilor RFID, armarea și dezarmarea alarmei și consultarea mesajelor de stare. Datele sunt transmise către aplicație prin brokerul MQTT, iar comenzile utilizatorului sunt trimise în sens invers, de la aplicație către ESP32.

Obiectivele lucrării sunt următoarele:

Obiectiv	Componentă folosită	Rezultat urmărit
Monitorizarea temperaturii și presiunii	Senzor BMP180	Afișare valorilor în aplicația PWA
Monitorizarea nivelului de lumină	Senzor TEMT6000	Stabilirea modului zi/noapte
Detectarea gazului	Senzor MQ135	Declanșarea unei alarme de gaz
Detectarea mișcării	Senzor PIR	Declanșarea unei alarme de mișcare
Controlul semnalizării vizuale	LED RGB	Schimbarea culorii din aplicație sau pe baza cardului RFID
Gestiunea accesului	Modul RFID RC522	Adăugarea, ștergerea și configurarea cardurilor
Comunicarea la distanță	MQTT și HiveMQ	Schimb de date între ESP32 și aplicația PWA
Afișarea datelor și comenzilor	Aplicație PWA	Interfață web pentru monitorizare și control

Tabel I.1. Obiectivele lucrării și modul de realizare

Metoda de lucru urmărită în această lucrare este orientată spre realizarea și testarea unui prototip funcțional, nu doar spre prezentarea teoretică a tehnologiilor utilizate. Din acest motiv, proiectul a fost abordat în mai multe etape, fiecare etapă având un rol clar în construirea sistemului final. Mai întâi au fost stabilite funcțiile principale ale sistemului și modul în care utilizatorul trebuie

să interacționeze cu acesta. Ulterior a fost proiectată arhitectura hardware, prin alegerea componentelor, stabilirea conexiunilor cu ESP32 și definirea rolului fiecărui senzor sau actuator.

După stabilirea arhitecturii hardware, etapa următoare a constat în implementarea firmware-ului pentru ESP32. Această parte include citirea senzorilor, controlul LED-ului RGB, al buzzerului și al LCD-ului, tratarea stărilor de alarmă, gestionarea cardurilor RFID și schimbul de mesaje MQTT cu brokerul. În paralel, a fost dezvoltată aplicația PWA, care oferă utilizatorului o interfață pentru monitorizarea valorilor, trimiterea comenzilor, configurarea cardurilor RFID și consultarea logurilor. Ultima etapă a constat în integrarea celor două componente, firmware și aplicație, și în testarea scenariilor principale de funcționare.

Lucrarea este organizată astfel încât să urmărească trecerea de la analiza inițială la implementarea practică. Primul capitol prezintă tehnologiile folosite în proiect, cu accent pe conceptele Smart Home, IoT, ESP32, MQTT și PWA. Al doilea capitol descrie cerințele sistemului, actorii implicați, cazurile de utilizare și structura topicurilor MQTT. Al treilea capitol este dedicat arhitecturii hardware, componentelor utilizate și schemei de conectare. Al patrulea capitol prezintă firmware-ul ESP32, de la inițializare și bucla principală până la senzorii, alarmele, RFID-ul și comunicația MQTT. Al cincilea capitol descrie aplicația PWA, ecranele acesteia și modul în care interfața comunică prin brokerul MQTT. Al șaselea capitol prezintă testarea sistemului, rezultatele obținute și limitele observate în urma validării prototipului.

Prin această structură, lucrarea urmărește să arate traseul complet al proiectului, de la idee și cerințe până la prototipul testat. Contribuția principală constă în integrarea componentelor hardware și software într-un sistem coerent, capabil să citească senzori, să comunice prin MQTT, să afișeze date într-o aplicație PWA și să execute comenzi trimise de utilizator. Astfel, proiectul demonstrează nu doar funcționarea unor componente separate, ci și modul în care acestea pot fi coordonate într-o arhitectură Smart Home.

Capitolul 1. Fundamente teoretice și tehnologii utilizate

1.1. Conceptul de Smart Home și rolul IoT

O locuință inteligentă poate fi privită ca un sistem în care obiectele și instalațiile dintr-un spațiu locuit nu mai funcționează complet separat, ci sunt legate prin senzori, elemente de comandă și o interfață software. Într-un astfel de sistem, valoarea nu este dată doar de existența unor dispozitive moderne, ci de modul în care acestea pot lucra împreună. Un senzor de lumină, un detector de mișcare, un senzor de gaz, un cititor RFID sau un buzzer devin relevante atunci când informațiile furnizate de ele sunt citite, interpretate și transformate într-un răspuns util pentru utilizator.

Prin comparație cu o instalație clasică, unde fiecare echipament este acționat separat, un sistem Smart Home creează o legătură între mediul fizic și o aplicație de control. Această legătură permite ca date precum temperatura, presiunea atmosferică, nivelul de lumină, detecția de mișcare sau valoarea unui senzor de gaz să fie urmărite într-o interfață digitală. Utilizatorul nu mai trebuie să verifice separat fiecare componentă, ci poate vedea starea generală a sistemului într-un singur loc.

În proiectul de față, această idee este aplicată prin conectarea mai multor senzori și module la placa ESP32. Microcontrolerul citește valorile din mediul fizic, le transmite prin MQTT și primește la rândul său comenzi din aplicația PWA. Din aceeași interfață, utilizatorul poate schimba culoarea LED-ului RGB, poate arma sau dezarma alarma, poate porni adăugarea unui card RFID și poate consulta mesajele de stare ale sistemului. Astfel, conceptul de locuință inteligentă este tratat aici printr-un prototip concret, nu doar ca noțiune teoretică.

Internet of Things explică trecerea de la o automatizare locală simplă la un sistem conectat, în care datele pot fi transmise, interpretate și folosite în mai multe puncte ale arhitecturii. Într-o instalație obișnuită, un senzor poate comanda direct un dispozitiv aflat lângă el, fără ca informația să fie disponibilă și în altă parte. Într-un sistem IoT, aceeași valoare poate fi citită de un microcontroler, transmisă prin rețea, afișată într-o aplicație și folosită pentru declanșarea unei comenzi. Recomandarea ITU-T Y.2060 descrie IoT ca o infrastructură prin care entități fizice și virtuale pot fi conectate cu ajutorul tehnologiilor de comunicație, pentru a susține servicii mai avansate[B1]. Această idee se regăsește direct în sistemele Smart Home, unde senzorii nu sunt folosiți doar pentru afișare locală, ci devin surse de date pentru o interfață de control.

Într-o abordare practică, IoT presupune existența unor dispozitive care combină componente hardware, firmware și conectivitate. Aceste dispozitive pot trimite date către alte sisteme sau pot primi instrucțiuni printr-o rețea. NIST include în această categorie senzori, controlere și aparate conectate la internet[RW1]. În lucrarea de față, acest rol este îndeplinit de ESP32, care face legătura dintre componentele fizice ale montajului și aplicația PWA. Placa nu este folosită doar pentru citirea unor

valori, ci și pentru interpretarea stării sistemului, publicarea mesajelor MQTT, primirea comenzilor și activarea elementelor locale de semnalizare.

În literatura de specialitate, sistemele Smart Home bazate pe IoT sunt descrise ca ansambluri în care senzori, actuatori și aplicații software lucrează împreună pentru automatizarea unor funcții din spațiul locuit[B2]. Această idee se regăsește și în proiectul de față, dar într-o formă adaptată unui prototip de licență. Sistemul nu urmărește acoperirea tuturor funcțiilor unei locuințe inteligente comerciale, ci integrarea unor componente accesibile într-o arhitectură clară: senzori pentru monitorizare, LED RGB și buzzer pentru semnalizare, RFID pentru identificare și aplicație PWA pentru interacțiunea cu utilizatorul.

Un avantaj important al unei locuințe conectate este faptul că utilizatorul nu trebuie să fie lângă dispozitiv pentru a verifica starea sistemului sau pentru a trimite o comandă. Într-o soluție strict locală, verificarea unei valori sau acționarea unui buton presupune contact direct cu echipamentul. În arhitectura propusă, aplicația PWA comunică prin brokerul MQTT cu ESP32, iar utilizatorul poate vedea valorile senzorilor, poate controla LED-ul RGB, poate arma sau dezarma alarma, poate adăuga ori șterge carduri RFID și poate consulta jurnalul de evenimente dintr-o singură interfață.

Această flexibilitate trebuie privită împreună cu riscurile aduse de conectarea dispozitivelor la rețea. Un sistem Smart Home poate transmite informații despre mediul din locuință și poate controla funcții care țin de siguranță, motiv pentru care securitatea comunicării, autentificarea și modul de stocare a datelor trebuie analizate încă din etapa de proiectare. Studiile privind utilizatorii de dispozitive Smart Home arată că percepțiile legate de securitate și confidențialitate diferă în funcție de categoria dispozitivului folosit[B3]. Pentru lucrarea de față, această observație este relevantă mai ales în cazul comunicației MQTT, al cardurilor RFID și al salvării setărilor aplicației în browser.

În această lucrare, conceptul de Smart Home este folosit într-un sens aplicat. Sistemul propus conectează senzori, elemente de semnalizare, un modul RFID și o aplicație PWA printr-o arhitectură bazată pe ESP32 și MQTT. Rolul IoT este acela de a lega aceste componente astfel încât datele să poată fi transmise, comenzile să poată fi executate, iar starea sistemului să poată fi urmărită într-o interfață accesibilă. Prin această abordare, proiectul oferă o bază practică pentru înțelegerea modului în care un prototip Smart Home poate fi construit pornind de la citirea senzorilor și ajungând până la afișarea informațiilor în aplicația utilizatorului.

1.2 Microcontrolere în sisteme Smart Home. Rolul ESP32

Într-un sistem Smart Home, microcontrolerul este componenta care face legătura dintre partea fizică a montajului și partea software prin care utilizatorul urmărește sau controlează sistemul. Senzorii pot furniza valori despre temperatură, lumină, gaz sau mișcare, iar actuatorii pot produce

semnale vizuale ori sonore, dar aceste elemente trebuie coordonate de o unitate capabilă să citească semnale, să execute instrucțiuni și să activeze ieșiri. Din acest motiv, microcontrolerul poate fi privit ca zona de decizie locală a unui prototip Smart Home.

Rolul microcontrolerului nu se oprește la citirea unor valori brute. O valoare analogică primită de la un senzor de gaz, de exemplu, devine utilă doar dacă firmware-ul o compară cu un prag și decide dacă trebuie declanșată o avertizare. La fel, o citire RFID nu are valoare practică dacă sistemul nu verifică UID-ul cardului, nu stabilește dacă acesta este cunoscut și nu execută o acțiune asociată. Prin aceste operații, microcontrolerul transformă semnalele electrice în comportamente concrete ale sistemului.

În proiectul de față, rolul de unitate centrală este îndeplinit de ESP32. Alegerea acestei platforme este justificată prin conectivitatea WiFi integrată și prin varietatea perifericelor disponibile. Documentația Espressif descrie ESP32 ca un cip care combină conectivitatea WiFi la 2,4 GHz cu Bluetooth, fiind conceput pentru aplicații care au nevoie de comunicație wireless și consum eficient[B4]. În această lucrare este folosită conexiunea WiFi, deoarece ESP32 trebuie să transmită date către brokerul MQTT și să primească mesaje de comandă de la aplicația PWA.

Un alt motiv pentru alegerea ESP32 este faptul că proiectul folosește componente cu moduri diferite de comunicare. MQ135 și TMT6000 sunt citiți prin intrări analogice, BMP180 comunică prin I2C, iar modulul RFID RC522 folosește SPI. Senzorul PIR și butonul fizic sunt tratate ca intrări digitale, iar LED-ul RGB și buzzerul sunt comandate prin ieșiri configurate în firmware. Această diversitate ar fi mai greu de gestionat cu o platformă mai limitată, în timp ce ESP32 permite integrarea lor într-un singur sistem. Manualul tehnic al ESP32 descrie aceste periferice ca parte a arhitecturii interne a cipului, ceea ce explică folosirea lui în proiecte cu senzori și module variate[B5].

În implementarea realizată, ESP32 funcționează ca un nod IoT local și contribuie la autonomia locală a sistemului. Acesta citește periodic senzorii, actualizează starea sistemului, gestionează alarma, tratează cardurile RFID și publică mesaje către brokerul MQTT. În sens invers, primește comenzi din aplicația PWA și le aplică asupra componentelor fizice. Astfel, datele nu ajung direct de la senzori la utilizator, ci trec printr-o etapă de prelucrare locală, în care firmware-ul decide ce trebuie afișat, ce trebuie transmis și ce reacție trebuie produsă.

Folosirea ESP32 simplifică și dezvoltarea proiectului, deoarece permite programarea într-un mediu accesibil și oferă compatibilitate cu biblioteci folosite frecvent în ecosistemul Arduino. Astfel, timpul necesar pentru inițializarea componentelor este redus, iar accentul poate fi pus pe integrarea senzorilor, pe comunicația MQTT și pe validarea scenariilor de utilizare. Pentru o lucrare de licență cu caracter aplicativ, această alegere este practică deoarece permite construirea unui prototip real, testabil și extensibil.

Prin urmare, ESP32 are un rol central în sistemul Smart Home implementat. El colectează date de la senzori, controlează elementele de avertizare, comunică prin WiFi cu brokerul MQTT și execută comenzile trimise din aplicația PWA. Importanța sa nu vine doar din faptul că este o placă de dezvoltare cu WiFi, ci din capacitatea de a reuni într-un singur punct partea hardware, firmware-ul și comunicația IoT.

1.3 Protocolul MQTT în aplicații IoT

Într-un sistem Smart Home, comunicarea dintre dispozitivul fizic și aplicația utilizatorului este la fel de importantă ca citirea senzorilor. Valorile măsurate de ESP32 trebuie să ajungă într-o interfață unde pot fi urmărite, iar comenzile introduse în aplicație trebuie să se întoarcă spre montajul hardware. Fără această legătură, sistemul ar rămâne o automatizare locală, capabilă să reacționeze doar în interiorul montajului, fără să transmită valori, stări sau alarme către utilizator. Din acest motiv, alegerea protocolului de comunicație influențează modul în care prototipul poate fi monitorizat și controlat.

Pentru această lucrare a fost ales MQTT, un protocol folosit frecvent în aplicații IoT datorită modului simplu în care organizează schimbul de mesaje. Specificația OASIS descrie MQTT ca protocol de transport bazat pe relația client-server și pe modelul publish/subscribe, potrivit pentru situații în care dispozitivele au resurse limitate sau traficul de rețea trebuie menținut redus[RW2]. Această logică se potrivește proiectului, deoarece ESP32 transmite periodic mesaje scurte, iar aplicația PWA primește doar informațiile publicate pe topicurile la care este abonată.

Modelul publish/subscribe diferă de comunicarea clasică de tip cerere-răspuns. Aplicația nu trebuie să întrebe permanent ESP32 dacă există valori noi, iar microcontrolerul nu trebuie să trimită date direct către o anumită pagină web. Ambele părți comunică prin broker. ESP32 publică un mesaj pe un topic, brokerul îl primește, iar apoi îl distribuie către clienții abonați. Această separare face ca aplicația PWA și microcontrolerul să nu depindă direct unul de celălalt, ci doar de structura topicurilor și de brokerul MQTT.

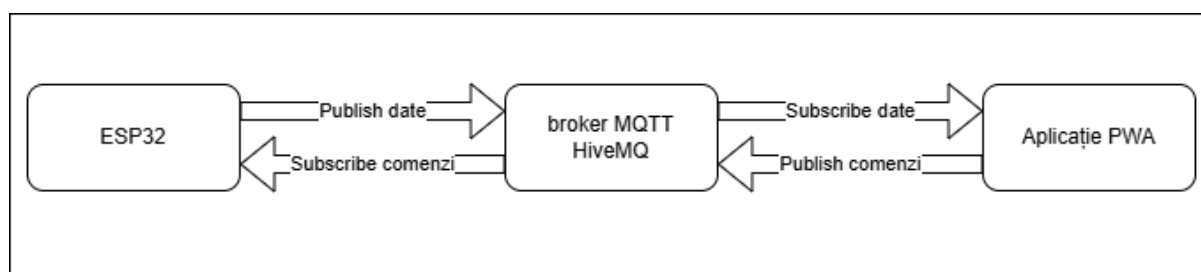


Figura 1.1 Modelul publish/subscribe în MQTT

Topicurile MQTT permit organizarea comunicației pe categorii clare de mesaje. Un topic poate fi privit ca o adresă logică prin care se separă datele de senzori, comenzile, alarmele,

evenimentele RFID sau logurile. În sistemul implementat, de exemplu, valorile de temperatură, presiune, lumină, gaz și mișcare sunt publicate pe un topic dedicat senzorilor, în timp ce comenzile pentru LED, alarmă sau RFID sunt trimise pe topicuri separate. Această împărțire face comunicația mai ușor de urmărit și permite extinderea sistemului prin adăugarea unor topicuri noi, fără modificarea întregii arhitecturi.

Conținutul efectiv al mesajelor este transmis prin payload, iar în proiect acesta este organizat în format JSON. Alegerea este practică, deoarece JSON permite gruparea mai multor câmpuri într-o structură ușor de interpretat atât de firmware, cât și de aplicația PWA. Un mesaj de senzori poate conține temperatura, presiunea, valoarea senzorului de gaz, starea PIR și starea alarmei, în timp ce o comandă pentru LED poate include valorile RGB. În acest fel, mesajele MQTT rămân scurte, dar suficient de clare pentru a descrie starea sistemului sau acțiunea cerută.

Un avantaj important al acestei soluții este că aplicația PWA nu depinde de o adresă IP locală fixă a ESP32. Ambele componente se conectează la brokerul MQTT, iar mesajele sunt schimbate prin topicuri. Pentru testarea prototipului a fost utilizată platforma HiveMQ Cloud, care permite folosirea unui broker cloud și oferă suport pentru conexiuni MQTT prin WebSocket securizat, necesare aplicațiilor rulate în browser[B6]. În aplicația PWA, conectarea la broker este realizată prin clientul Eclipse Paho JavaScript, care permite publicarea și recepționarea mesajelor MQTT dintr-o pagină web[RW3].

Comparativ cu o comunicare bazată pe HTTP, MQTT este mai potrivit pentru acest prototip deoarece nu obligă aplicația să trimită cereri repetate pentru a verifica dacă există date noi. ESP32 publică valorile atunci când trebuie transmise, iar brokerul le distribuie către aplicația abonată. Această funcționare se potrivește mai bine unui sistem în care senzorii trimit periodic date, iar interfața trebuie să se actualizeze fără interogări continue.

În această lucrare, MQTT are rolul de legătură principală între firmware și aplicația PWA. Prin acest protocol, datele citite din mediul fizic ajung în interfața utilizatorului, iar acțiunile realizate în aplicație sunt transformate în comenzi executate de ESP32. Astfel, arhitectura separă clar nivelul hardware, brokerul de mesaje și interfața de control, ceea ce face sistemul mai ușor de explicat, testat și extins.

1.4 Aplicațiile PWA pentru controlul sistemelor IoT

Aplicațiile PWA, denumite Progressive Web Apps, reprezintă o soluție între aplicațiile web obișnuite și aplicațiile native instalate pe dispozitive mobile. Ele sunt dezvoltate cu tehnologii web, precum HTML, CSS și JavaScript, dar pot oferi o experiență mai apropiată de cea a unei aplicații dedicate, prin adaptarea la dimensiunea ecranului, posibilitatea de instalare și integrarea cu anumite funcții ale browserului[RW4]. În contextul unui sistem IoT, această abordare este utilă deoarece

permite realizarea unei interfețe de control accesibile rapid, fără dezvoltarea separată a unor aplicații native pentru Android sau iOS.

În proiectul de față, aplicația PWA are rolul de panou de control pentru sistemul Smart Home bazat pe ESP32. Interfața este împărțită în mai multe ecrane: dashboard pentru valorile senzorilor și starea alarmei, ecran pentru controlul LED-ului RGB, secțiune RFID pentru adăugarea și ștergerea cardurilor, zonă de configurare pentru comportamentul fiecărui card și pagină pentru loguri și setări MQTT. Prin această organizare, utilizatorul poate vedea starea sistemului și poate trimite comenzi fără să folosească monitorul serial sau un client MQTT separat.

Aplicația nu este doar o pagină de afișare, ci un punct de interacțiune bidirecțională. Ea primește date de la ESP32 prin brokerul MQTT și actualizează interfața în funcție de mesaje primite. În același timp, atunci când utilizatorul apasă un buton sau modifică o valoare, aplicația publică un mesaj de comandă. De exemplu, schimbarea culorii LED-ului RGB pornește dintr-un control vizual din interfață, dar ajunge la ESP32 sub forma unui payload MQTT. În același mod, armarea alarmei sau configurarea unui card RFID sunt acțiuni începute în aplicație și executate ulterior de firmware.

Un element specific aplicațiilor PWA este fișierul manifest, prin care browserul primește informații despre numele aplicației, pictograme, culori și modul de lansare. Acest mecanism ajută aplicația să fie prezentată într-o formă apropiată de o aplicație instalabilă, mai ales pe dispozitive mobile[RW5]. În proiectul analizat, interfața este deja orientată spre utilizare mobilă prin dimensiunea elementelor, navigarea inferioară și organizarea pe ecrane. Adăugarea unui manifest complet ar putea consolida caracterul de PWA și ar permite o integrare mai bună pe dispozitivul utilizatorului.

Un alt element asociat frecvent aplicațiilor PWA este service worker-ul. Acesta poate funcționa între aplicație, browser și rețea, permițând gestionarea unor resurse, încărcarea mai rapidă a interfeței și anumite funcții offline[RW6]. În cazul unui sistem Smart Home, funcționarea offline trebuie privită cu atenție, deoarece monitorizarea în timp real și comenzile către ESP32 depind de conexiunea cu brokerul MQTT. Totuși, într-o versiune extinsă, un service worker ar putea fi folosit pentru păstrarea resurselor statice în cache și pentru afișarea mai clară a stării deconectate.

Prin rolul său, aplicația PWA transformă sistemul Smart Home într-o soluție accesibilă utilizatorului final. ESP32 citește senzori și execută comenzi, dar aplicația este cea care face aceste funcții ușor de urmărit și controlat. În arhitectura propusă, PWA-ul devine punctul de contact dintre utilizator și infrastructura IoT, justificând includerea sa ca element central al proiectului.

1.5 Brokerul cloud HiveMQ și comunicația prin WebSocket

Într-o arhitectură IoT bazată pe MQTT, brokerul este componenta care primește mesajele publicate de clienți și le distribuie mai departe către cei abonați la topicurile respective. ESP32, aplicația PWA și

eventuale alte aplicații viitoare nu trebuie să comunice direct între ele, ci folosesc brokerul ca punct comun de schimb al mesajelor. Pentru un sistem Smart Home, această organizare este utilă deoarece separă dispozitivul fizic de interfața software și permite extinderea sistemului fără modificarea completă a arhitecturii.

În proiectul de față a fost utilizat HiveMQ Cloud, în varianta gratuită, ca broker MQTT găzduit online. Alegerea unui broker cloud simplifică testarea, deoarece nu mai este necesară configurarea unui broker local pe un calculator sau server aflat permanent în funcțiune. Clienții MQTT se conectează la același serviciu online folosind datele de autentificare, iar schimbul de mesaje nu mai depinde strict de adrese IP locale sau de configurarea routerului. Pentru un prototip de licență, această soluție este suficient de accesibilă și permite concentrarea pe integrarea dintre ESP32, MQTT și aplicația PWA[RW8].

În cazul aplicației PWA, conexiunea la broker se realizează prin WebSocket securizat. Browserul nu deschide în mod obișnuit conexiuni MQTT TCP brute, așa cum poate face un firmware sau o aplicație nativă, ci folosește mecanisme disponibile în mediul web. WebSocket API permite stabilirea unei sesiuni de comunicare bidirecțională între aplicația web și server, ceea ce este potrivit pentru o interfață Smart Home care trebuie să primească date în timp real și să trimită comenzi către sistem[RW3]. În configurația folosită cu HiveMQ Cloud, aplicația PWA utilizează conexiunea WebSocket securizată pe portul corespunzător.

Pentru implementarea clientului MQTT în browser a fost folosită biblioteca Eclipse Paho JavaScript Client. Aceasta permite unei aplicații web să se conecteze la un broker MQTT prin WebSocket, să se aboneze la topicuri și să publice mesaje[RW9]. În proiect, Paho este folosit pentru recepționarea valorilor de senzori, a alarmelor, a evenimentelor RFID și a logurilor, dar și pentru trimiterea comenzilor de control către ESP32. Astfel, aplicația PWA se comportă ca un client MQTT complet, adaptat mediului web.

Prin folosirea HiveMQ Cloud și WebSocket, proiectul poate conecta ESP32 cu aplicația PWA fără construirea unui server propriu. Brokerul asigură schimbul de mesaje MQTT, iar WebSocket permite aplicației din browser să participe la această comunicație. Împreună, aceste elemente oferă o soluție accesibilă pentru un prototip Smart Home bazat pe IoT, control web și comunicație bidirecțională.

1.6 Considerații privind securitatea

Securitatea unui sistem Smart Home trebuie avută în vedere încă din etapa de proiectare, deoarece dispozitivele conectate pot transmite informații despre locuință și pot executa acțiuni asupra unor componente fizice. Un senzor de mișcare, un modul RFID sau o alarmă de gaz par elemente simple, dar datele și comenzile asociate lor pot influența direct modul în care utilizatorul percepe

siguranța sistemului. Într-o arhitectură IoT, riscurile nu apar doar la nivelul dispozitivului, ci și în rețea, în brokerul MQTT, în aplicația web și în modul de gestionare a datelor de autentificare.

NIST propune pentru dispozitivele IoT o abordare bazată pe capabilități precum identificarea dispozitivului, configurarea controlată, protejarea datelor, accesul logic la interfețe și posibilitatea de actualizare a software-ului[RW4]. Chiar dacă sistemul realizat este un prototip academic, aceste direcții sunt utile pentru stabilirea unor limite. ESP32 trebuie să aibă un client MQTT distinct, mesajele trebuie să respecte o structură previzibilă, iar comenzile trebuie interpretate numai după verificarea payload-ului. De exemplu, ștergerea unui card RFID trebuie să conțină un UID valid, iar controlul LED-ului trebuie să primească valori RGB acceptabile.

Autentificarea la broker este una dintre zonele importante. Dacă un client neautorizat ar putea publica mesaje pe topicurile sistemului, acesta ar putea trimite comenzi către ESP32 sau ar putea citi date despre starea locuinței. OWASP menționează parolele slabe, serviciile de rețea nesigure și transferul sau stocarea neprotejată a datelor printre problemele frecvente din ecosistemele IoT[B7]. Din acest motiv, chiar și într-un prototip, este necesară folosirea unor credențiale dedicate și evitarea parolilor evidente.

Specificația MQTT arată modul în care clienții publică și primesc mesaje, dar securitatea finală depinde de implementare, configurație și broker[RW10]. Protocolul nu elimină automat riscurile legate de autentificare, autorizare sau protecția transportului. În sistemul propus, această observație este importantă deoarece topicurile de comandă pot modifica starea alarmei, culoarea LED-ului sau lista cardurilor RFID. Prin urmare, topicurile trebuie organizate clar, iar accesul la ele ar trebui limitat în funcție de rolul fiecărui client.

O altă limitare ține de aplicația PWA. În prototip, hostul brokerului, portul, utilizatorul și parola sunt salvate local în browser pentru a simplifica reconectarea. Soluția este practică în etapa de testare, dar nu este ideală pentru un sistem folosit în condiții reale. Pentru o versiune mai sigură, stocarea locală a credențialelor ar trebui înlocuită sau completată cu un backend propriu, tokenuri temporare ori reguli mai stricte de acces la topicuri.

Prin aceste considerații, securitatea nu este tratată ca o funcție separată, adăugată la final, ci ca o condiție care influențează modul de proiectare al sistemului. Pentru lucrarea de față, obiectivul principal rămâne realizarea unui prototip funcțional, însă identificarea riscurilor arată direcțiile în care sistemul poate fi îmbunătățit. Astfel, proiectul demonstrează funcționarea unei arhitecturi Smart Home bazate pe ESP32, MQTT și PWA, dar evidențiază și limitele care trebuie gestionate înainte de o utilizare într-un mediu real.

Capitolul 2. Analiza cerințelor și proiectarea funcțională

2.1. Descrierea studiului de caz

Studiul de caz al lucrării constă în proiectarea și implementarea unui prototip Smart Home bazat pe ESP32, senzori de mediu, modul RFID, elemente de semnalizare locală și o aplicație PWA conectată prin MQTT la brokerul HiveMQ Cloud. Sistemul modelează, la scară experimentală, o locuință inteligentă în care utilizatorul poate urmări anumite condiții din spațiul locuit și poate trimite comenzi către dispozitivul fizic printr-o interfață web accesibilă de pe telefon sau calculator.

Scenariul de la care pornește proiectul este cel al unei camere care are nevoie de monitorizare de bază și de control simplu la distanță. Utilizatorul poate urmări temperatura, presiunea atmosferică, nivelul de lumină, detecția de mișcare și valoarea unui senzor de gaz. În același timp, poate arma sau dezarma alarma, poate controla un LED RGB și poate gestiona carduri RFID. Aceste funcții au fost reunite într-un prototip care arată cum pot colabora componente hardware și software într-un sistem Smart Home.

Studiul de caz are caracter aplicativ și urmărește validarea unei soluții funcționale, nu realizarea unui produs comercial certificat. Prototipul permite testarea modului în care un microcontroler poate integra senzori, actuatoare, RFID și comunicație cloud într-un sistem coerent. În același timp, el evidențiază și limitele unei astfel de soluții, precum dependența de conexiunea la internet, securitatea datelor de conectare, acuratețea senzorilor și necesitatea calibrării unor componente.

Prin urmare, studiul de caz propus reflectă un scenariu Smart Home complet la nivel de prototip: monitorizarea mediului, controlul local și la distanță, semnalizarea evenimentelor, identificarea RFID și comunicarea prin MQTT. Această structură oferă baza pentru definirea cerințelor, descrierea arhitecturii, explicarea firmware-ului, prezentarea aplicației PWA și evaluarea rezultatelor obținute prin testare.

2.2. Actorii sistemului

Pentru descrierea funcțională a sistemului Smart Home, trebuie identificați actorii care participă la schimbul de date, la transmiterea comenzilor și la declanșarea evenimentelor monitorizate. În această lucrare, termenul de actor este folosit într-un sens practic și include atât utilizatorul, cât și elementele tehnice care interacționează cu sistemul. Această abordare este potrivită deoarece proiectul nu este format dintr-o aplicație izolată, ci dintr-un ansamblu de componente hardware, servicii de comunicație și interfețe software.

Actorul principal este utilizatorul. Acesta interacționează cu sistemul prin aplicația PWA și, în anumite situații, prin cardul RFID. Din aplicație, utilizatorul poate urmări valorile senzorilor, poate verifica starea alarmei, poate controla LED-ul RGB, poate solicita logurile și poate configura cardurile RFID. Prin apropierea unui card de cititorul RC522, utilizatorul declanșează un proces local de identificare, tratat de firmware-ul ESP32. Astfel, accesul la funcțiile sistemului nu se face direct asupra fiecărei componente hardware, ci prin interfețe mai simple.

Aplicația PWA este actorul software prin care utilizatorul controlează sistemul și primește informații despre starea acestuia. Ea afișează datele publicate de ESP32, organizează informațiile în ecrane funcționale și transformă acțiunile utilizatorului în mesaje MQTT. De exemplu, la apăsarea butonului de armare, aplicația publică un mesaj pe topicul corespunzător, iar ESP32 îl primește prin broker și modifică starea alarmei în firmware. Prin această logică, aplicația are rol de interfață de prezentare și comandă, fără să preia funcțiile microcontrolerului.

ESP32 este actorul tehnic central al sistemului. Acesta citește senzorii, tratează cardurile RFID, controlează LED-ul RGB, buzzerul și LCD-ul, publică mesaje MQTT și execută comenzile primite din aplicație. În raport cu aplicația PWA, ESP32 este atât sursă de date, cât și receptor de comenzi. În raport cu senzorii și actuatorii, el este elementul care citește, interpretează și comandă. Această poziție îl face responsabil pentru comportamentul local al prototipului, inclusiv pentru reacții precum alarma de gaz sau semnalizarea mișcării.

Brokerul HiveMQ Cloud este actorul de comunicație dintre ESP32 și aplicația PWA. El nu interpretează logica alarmei, nu verifică UID-uri RFID și nu calculează valori de senzori, ci distribuie mesajele publicate pe topicuri către clienții abonați. Prin folosirea brokerului, aplicația nu trebuie să se conecteze direct la ESP32, iar ESP32 nu trebuie să cunoască locul în care rulează aplicația. Ambele componente comunică prin același punct intermediar, ceea ce simplifică arhitectura MQTT.

Cardul RFID poate fi considerat un actor extern de identificare. El nu rulează logică software și nu comunică direct cu brokerul, dar influențează sistemul atunci când este apropiat de cititorul RC522. UID-ul cardului este citit de ESP32, iar firmware-ul decide dacă acesta este cunoscut, dacă trebuie adăugat, șters sau folosit pentru aplicarea unei configurații. În studiul de caz, RFID-ul este folosit pentru a demonstra o formă simplă de identificare și personalizare a comportamentului sistemului.

Mediul monitorizat este un actor pasiv, dar important. Acesta include condițiile fizice din spațiul supravegheat: temperatura, presiunea, lumina, gazul și mișcarea. Mediul nu trimite comenzi software, însă modificările sale sunt detectate de senzori și devin evenimente tratate de ESP32. De exemplu, o creștere a valorii senzorului MQ135 poate declanșa alarma de gaz, iar mișcarea detectată de PIR poate activa logica de alarmă dacă sistemul este armat.

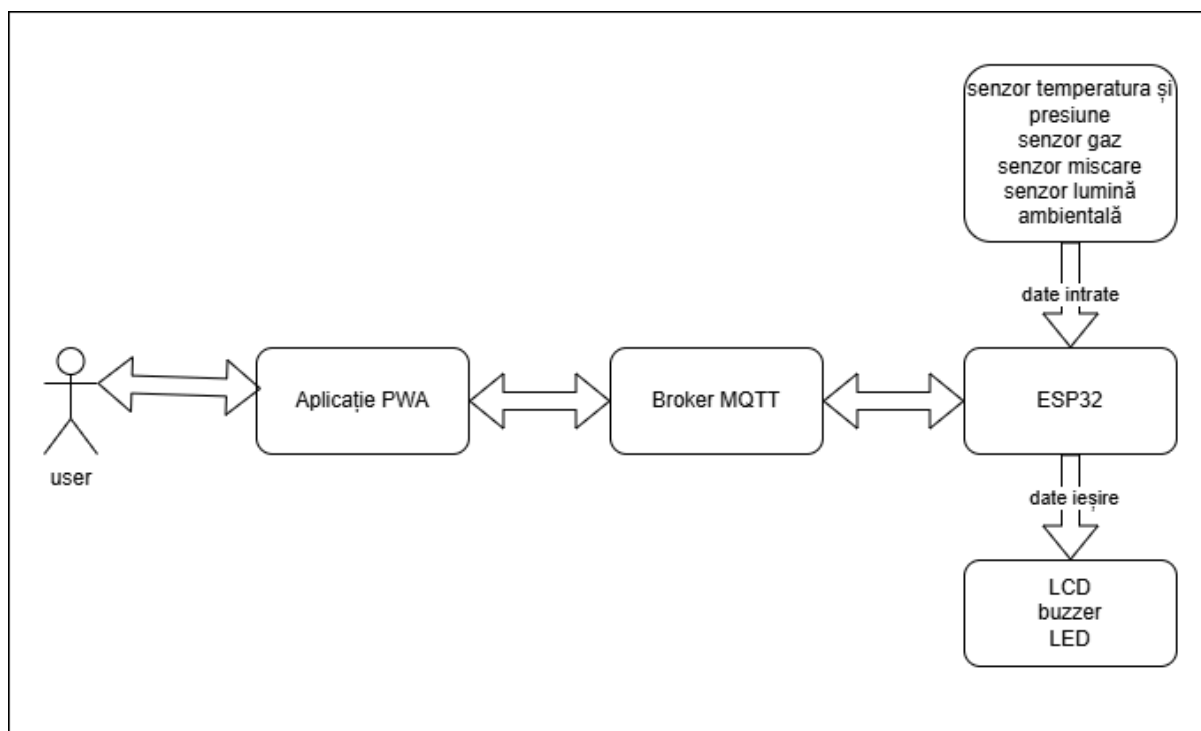


Figura 2.1 Diagrama actorilor sistemului

Identificarea actorilor ajută la clarificarea modului în care sistemul este folosit și la delimitarea responsabilităților fiecărei componente. Utilizatorul inițiază acțiuni și interpretează informații, aplicația PWA transformă acțiunile în mesaje MQTT, brokerul distribuie mesajele, ESP32 execută logica de control, cardul RFID furnizează o identitate locală, iar mediul monitorizat generează valorile care stau la baza funcțiilor de alarmare și afișare. Această împărțire pregătește definirea cerințelor funcționale, deoarece fiecare cerință poate fi raportată la unul sau mai mulți actori ai sistemului.

2.3. Cazuri de utilizare

Cazurile de utilizare descriu modul în care actorii identificați anterior interacționează cu sistemul Smart Home pentru a obține un rezultat concret. Ele pornesc de la perspectiva utilizatorului sau a unei componente externe și prezintă acțiunile principale care trebuie susținute de prototip. În cadrul acestei lucrări, cazurile de utilizare nu sunt formulate doar ca scenarii generale, ci sunt corelate cu funcțiile reale ale sistemului: monitorizarea senzorilor, controlul LED-ului RGB, armarea alarmei, gestionarea cardurilor RFID, transmiterea comenzilor prin MQTT și afișarea logurilor în aplicația PWA.

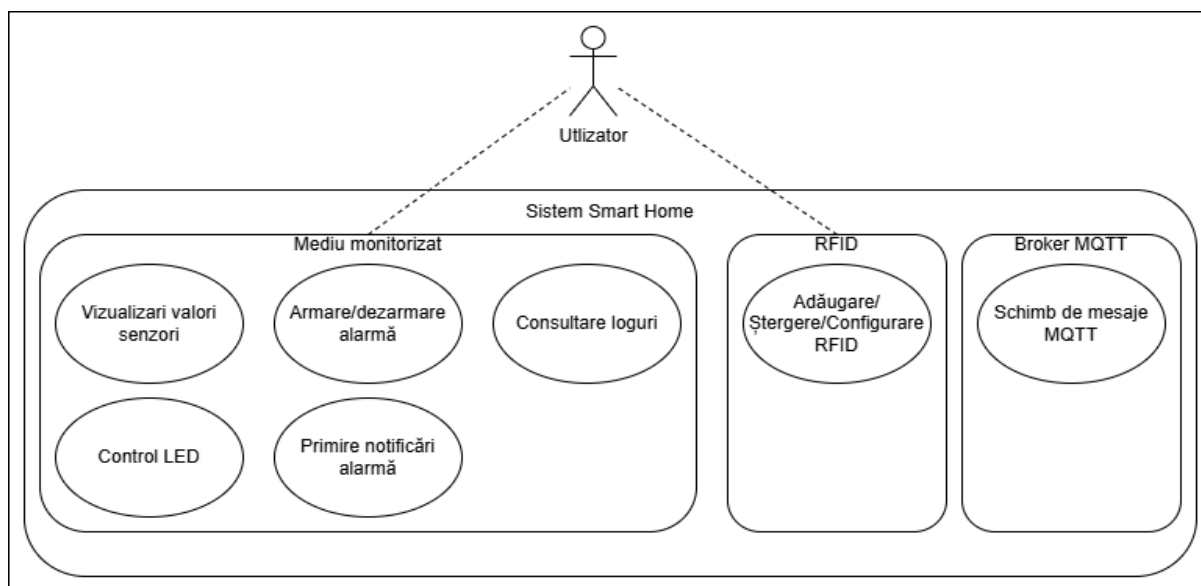


Figura 2.2 Diagrama cazurilor de utilizare

Primul caz de utilizare este monitorizarea senzorilor. ESP32 citește periodic valorile primite de la componentele conectate și le publică prin MQTT, iar aplicația PWA le afișează în dashboard. Utilizatorul nu trebuie să pornească manual fiecare citire, deoarece această activitate este gestionată de firmware. Rolul său este să urmărească interfața și să interpreteze datele afișate. Acest caz de utilizare confirmă funcția de bază a prototipului: transformarea valorilor din mediul fizic în informații vizibile într-o aplicație web.

Al doilea caz de utilizare este armarea și dezarmarea alarmei. Utilizatorul apasă butonul corespunzător din aplicația PWA, iar aplicația publică o comandă MQTT către ESP32. Firmware-ul modifică starea internă a alarmei și transmite ulterior această stare în mesajele de status. Scenariul arată comunicarea în sens invers față de monitorizarea senzorilor, deoarece acțiunea pornește de la utilizator, trece prin aplicație și broker, apoi ajunge la microcontroler.

Al treilea caz de utilizare este declanșarea alarmei de gaz. Senzorul MQ135 transmite o valoare analogică, iar ESP32 o compară cu pragul stabilit în firmware. Dacă valoarea indică o posibilă situație de risc, sistemul activează semnalizarea locală și publică evenimentul către aplicația PWA. Utilizatorul poate observa starea în dashboard și în loguri, iar prototipul oferă feedback prin LED sau buzzer. Acest caz arată că sistemul poate reacționa automat la modificări din mediul monitorizat.

Al patrulea caz de utilizare privește alarma de mișcare. Senzorul PIR detectează mișcarea, iar ESP32 verifică dacă alarma este armată. Dacă sistemul se află în această stare, firmware-ul tratează mișcarea ca eveniment de securitate și transmite informația către aplicație. În funcție de logica implementată, sistemul poate semnala o stare de avertizare sau o alarmă activă. Scenariul leagă direct un senzor digital, starea internă a sistemului și notificarea utilizatorului.

Al cincilea caz de utilizare este controlul LED-ului RGB. Utilizatorul alege în aplicație valorile pentru canalele roșu, verde și albastru, iar aplicația le trimite către ESP32 prin topicul MQTT de comandă. Microcontrolerul interpretează mesajul și modifică semnalul PWM aplicat LED-ului. Rezultatul este vizibil imediat prin schimbarea culorii. Acest caz demonstrează controlul unui actuator fizic dintr-o interfață web și confirmă comunicarea bidirecțională dintre aplicație și dispozitiv.

Al șaselea caz de utilizare este adăugarea unui card RFID. Utilizatorul pornește din aplicație modul de adăugare, apoi apropie cardul de cititorul RC522. ESP32 citește UID-ul, îl memorează și transmite aplicației un mesaj pentru actualizarea listei de carduri. În acest caz, utilizatorul interacționează atât cu aplicația PWA, cât și cu dispozitivul fizic. Scenariul este relevant deoarece arată cum sunt combinate o comandă software, o acțiune locală și stocarea persistentă a unui identificator RFID.

Al șaptelea caz de utilizare este ștergerea unui card RFID. Utilizatorul introduce sau selectează UID-ul cardului în aplicație și trimite comanda de ștergere. ESP32 primește mesajul, caută cardul în memoria locală și elimină înregistrarea corespunzătoare. La o citire ulterioară, acel card nu mai trebuie tratat ca valid. Acest caz completează gestiunea RFID, deoarece un sistem care permite adăugarea identificatorilor trebuie să permită și eliminarea lor.

Al optulea caz de utilizare este configurarea comportamentului asociat unui card RFID. Utilizatorul introduce UID-ul și stabilește parametri precum culoarea LED-ului, folosirea LED-ului ziua sau noaptea și activarea buzzerului la citire. Aplicația transmite aceste date către ESP32, iar firmware-ul le salvează pentru a le aplica la următoarea citire a cardului. Prin această funcție, prototipul poate reacționa diferit în funcție de cardul utilizat.

Al nouălea caz de utilizare este consultarea logurilor. Utilizatorul solicită din aplicație evenimentele înregistrate, iar ESP32 transmite lista disponibilă prin MQTT. Aplicația afișează logurile într-o zonă dedicată, astfel încât utilizatorul poate verifica acțiunile recente, alarmele și comenzile importante. Această funcție este utilă mai ales în testare, deoarece permite urmărirea comportamentului sistemului după fiecare eveniment.

Cazurile de utilizare prezentate stau la baza proiectării topicurilor MQTT și a scenariilor de testare. Fiecare caz poate fi transformat într-o succesiune clară de pași, cu un rezultat așteptat care poate fi verificat în aplicație sau la nivelul componentelor hardware. În acest fel, analiza funcțională rămâne legată de implementarea reală a prototipului și pregătește definirea contractului de comunicație dintre ESP32 și aplicația PWA.

2.4. Contractul de comunicație MQTT

Contractul de comunicație MQTT stabilește modul în care ESP32, brokerul HiveMQ Cloud și aplicația PWA schimbă mesaje în cadrul sistemului Smart Home. Acesta nu este o componentă software separată, ci o regulă de organizare a comunicației: ce topicuri se folosesc, în ce direcție circulă mesajele, cum arată payload-urile și ce rol are fiecare mesaj. Prin definirea acestui contract, legătura dintre firmware și aplicația web devine mai ușor de urmărit, iar fiecare funcție a sistemului poate fi verificată printr-un flux clar de mesaje.

În arhitectura proiectului, topicurile folosesc prefixul comun smarthome, pentru ca mesajele sistemului să fie grupate logic. Datele trimise de ESP32 către aplicație sunt publicate pe topicuri pentru senzori, alarme, RFID, loguri și status. În schimb, comenzile trimise de aplicația PWA către ESP32 sunt grupate sub smarthome/cmd. Această separare arată clar diferența dintre mesajele de monitorizare și mesajele de comandă. Practic, aplicația se abonează la topicurile de stare și publică pe topicurile de comandă, iar ESP32 publică datele citite și ascultă comenzile pe care trebuie să le execute.

Payload-urile sunt construite în principal în format JSON, deoarece permit transmiterea mai multor informații într-un singur mesaj. De exemplu, mesajul pentru senzori poate include temperatura, presiunea, lumina, valoarea senzorului de gaz, starea alarmei și starea senzorului PIR. Pentru LED-ul RGB, aplicația poate trimite într-un singur mesaj valorile pentru roșu, verde și albastru. Folosirea JSON este utilă deoarece atât firmware-ul, cât și aplicația PWA pot interpreta datele după numele câmpurilor, iar structura poate fi extinsă ulterior fără schimbarea întregului mecanism de comunicație.

Tabelul 2.3. Contractul de comunicație MQTT al sistemului Smart Home (1/3)

Topic MQTT	Direcție	Publisher	Subscriber	Tip payload	Rol în sistem
smarthome /sensors	ESP32 către PWA	ESP32	PWA	JSON: temp, pres, light, gas, isDay, armed, gasAlarm, movAlarm, pir	Transmite periodic valorile senzorilor și starea generală a sistemului.

Tabelul 2.3. Contractul de comunicație MQTT al sistemului Smart Home (2/3)

Topic MQTT	Direcție	Publisher	Subscriber	Tip payload	Rol în sistem
smarthome /alarm/gas	ESP32 către PWA	ESP32	PWA	JSON: event, adc	Anunță declanșarea sau revenirea alarmei de gaz.
smarthome /alarm/motion	ESP32 către PWA	ESP32	PWA	JSON: event, silent	Anunță detectarea mișcării și starea alarmei de mișcare.
smarthome /rfid/new	ESP32 către PWA	ESP32	PWA	JSON: event, uid, index	Transmite aplicației informația că un card RFID nou a fost adăugat.
smarthome /log	ESP32 către PWA	ESP32	PWA	JSON: total, from, log	Transmite loguri în loturi, pentru afișare în aplicație.
smarthome /status	ESP32 către PWA	ESP32	PWA	JSON: status, ip, cards sau event, uid	Transmite mesaje de stare, conectare, acces permis, acces refuzat sau ștergere card.
smarthome /cmd/light	PWA către ESP32	PWA	ESP32	JSON: r, g, b sau on	Controlează LED-ul RGB sau îl stinge.
smarthome /cmd/alarm	PWA către ESP32	PWA	ESP32	JSON: armed	Armează sau dezarmează alarma.

Tabelul 2.3. Contractul de comunicație MQTT al sistemului Smart Home (3/3)

Topic MQTT	Direcție	Publisher	Subscriber	Tip payload	Rol în sistem
smarthome/cmd/rfid/add	PWA către ESP32	PWA	ESP32	Text simplu sau comandă scurtă	Activează modul de așteptare pentru adăugarea unui card RFID.
smarthome/cmd/rfid/delete	PWA către ESP32	PWA	ESP32	JSON: uid	Solicită ștergerea unui card RFID pe baza UID-ului.
smarthome/cmd/card/config	PWA către ESP32	PWA	ESP32	JSON: uid, r, g, b, ledOnDay, ledOnNight, buzzerOnRead	Salvează configurația asociată unui card RFID.
smarthome/cmd/log	PWA către ESP32	PWA	ESP32	Text simplu: request	Solicită trimiterea logurilor stocate local de ESP32.
smarthome/cmd/screen	PWA către ESP32 sau client MQTT extern	PWA, client de test	ESP32	JSON: line1, line2, duration	Permite afișarea temporară a unui mesaj pe LCD, fiind o comandă suportată de firmware.

Topicurile prezentate în tabel acoperă fluxurile principale ale prototipului. Mesajele smarthome/sensors sunt cele mai frecvente, deoarece ESP32 le publică periodic pentru actualizarea dashboard-ului din aplicație. Payload-ul conține atât valori numerice, precum temperatura, presiunea, lumina și valoarea senzorului de gaz, cât și stări logice, cum ar fi alarma sau detectarea mișcării. Prin

gruparea acestor informații într-un singur mesaj, aplicația poate actualiza mai multe zone ale interfeței fără să aștepte câte un mesaj separat pentru fiecare senzor.

Topicurile de alarmă sunt folosite pentru evenimente punctuale, nu pentru raportări periodice. Când valoarea citită de MQ135 depășește pragul stabilit, ESP32 publică un mesaj pe `smarthome/alarm/gas`, iar aplicația poate afișa avertizarea și poate înregistra evenimentul în loguri. Pentru mișcare, topicul `smarthome/alarm/motion` transmite informații despre evenimentul detectat de PIR și despre starea alarmei. Astfel, aplicația poate diferenția între simple valori de monitorizare și situații care cer atenție.

Topicurile de comandă sunt folosite atunci când utilizatorul modifică sistemul din aplicație. Pentru LED-ul RGB, aplicația publică pe `smarthome/cmd/light` un payload cu valorile canalelor roșu, verde și albastru, iar ESP32 le aplică prin semnale PWM. Pentru alarmă, aplicația trimite o valoare pentru câmpul `armed`, iar firmware-ul schimbă starea internă a sistemului. Aceste mesaje trebuie tratate cu atenție, deoarece nu sunt doar informative, ci produc efecte directe asupra dispozitivului fizic.

Pentru funcțiile RFID sunt necesare mai multe topicuri, deoarece operațiile sunt diferite. Adăugarea unui card începe printr-o comandă care activează modul de așteptare, iar citirea efectivă se face local prin RC522. După memorarea cardului, ESP32 publică mesajul pe `smarthome/rfid/new`, pentru ca aplicația să afișeze UID-ul și poziția cardului. Ștergerea și configurarea cardurilor folosesc payload-uri JSON, deoarece trebuie transmise UID-ul și, în cazul configurării, parametrii pentru LED și buzzer.

Logurile și mesajele de status au rol de feedback și diagnostic. Prin `smarthome/log`, aplicația primește evenimentele memorate de ESP32, iar prin `smarthome/status` sunt transmise mesaje scurte despre conectare, acces RFID sau rezultatul unor comenzi. Aceste topicuri sunt utile mai ales în etapa de testare, deoarece ajută la urmărirea reacției firmware-ului după acțiunile utilizatorului sau după evenimentele detectate de senzori.

Pentru ca acest contract MQTT să funcționeze corect, fiecare client trebuie să respecte structura mesajelor. Aplicația PWA trebuie să trimită exact câmpurile așteptate de firmware, iar ESP32 trebuie să publice mesaje pe care interfața le poate interpreta. De exemplu, o comandă de ștergere RFID trebuie să conțină un UID valid, iar o comandă de configurare trebuie să includă atât UID-ul, cât și valorile asociate cardului. Această organizare reduce erorile și face mai simplă testarea fiecărei funcții.

Contractul MQTT definește, astfel, limbajul comun dintre partea hardware și aplicația web. Prin topicuri clare, payload-uri previzibile și direcții bine stabilite, sistemul poate transmite date, comenzi și evenimente fără ambiguități.

Capitolul 3. Arhitectura hardware a sistemului

3.1. Vedere de ansamblu asupra sistemului hardware

Sistemul hardware realizat în această lucrare este construit în jurul plăcii ESP32, care are rolul de unitate centrală pentru citirea datelor, prelucrarea lor și comanda componentelor conectate. Sensorii, modulele de identificare și elementele de semnalizare sunt legate la această placă, iar firmware-ul stabilește cum sunt citite valorile, cum sunt tratate evenimentele și ce informații sunt trimise către aplicația PWA. Astfel, partea hardware nu este doar o alăturare de componente, ci un ansamblu în care fiecare element contribuie la monitorizarea sau controlul prototipului Smart Home.

La nivel general, componentele pot fi împărțite în patru grupe. Prima grupă include senzorii pentru monitorizarea mediului: BMP180 pentru temperatură și presiune, TEMT6000 pentru lumină, MQ135 pentru variații asociate gazului și PIR BISS0001 pentru mișcare. A doua grupă este formată din elementele de feedback local, adică LED-ul RGB, buzzerul activ și afișajul LCD 1602. A treia grupă este reprezentată de modulul RFID RC522, folosit pentru citirea cardurilor și pentru asocierea lor cu anumite comportamente ale sistemului. A patra grupă ține de comunicație, realizată prin WiFi-ul integrat al ESP32 și prin mesaje MQTT transmise către brokerul cloud.

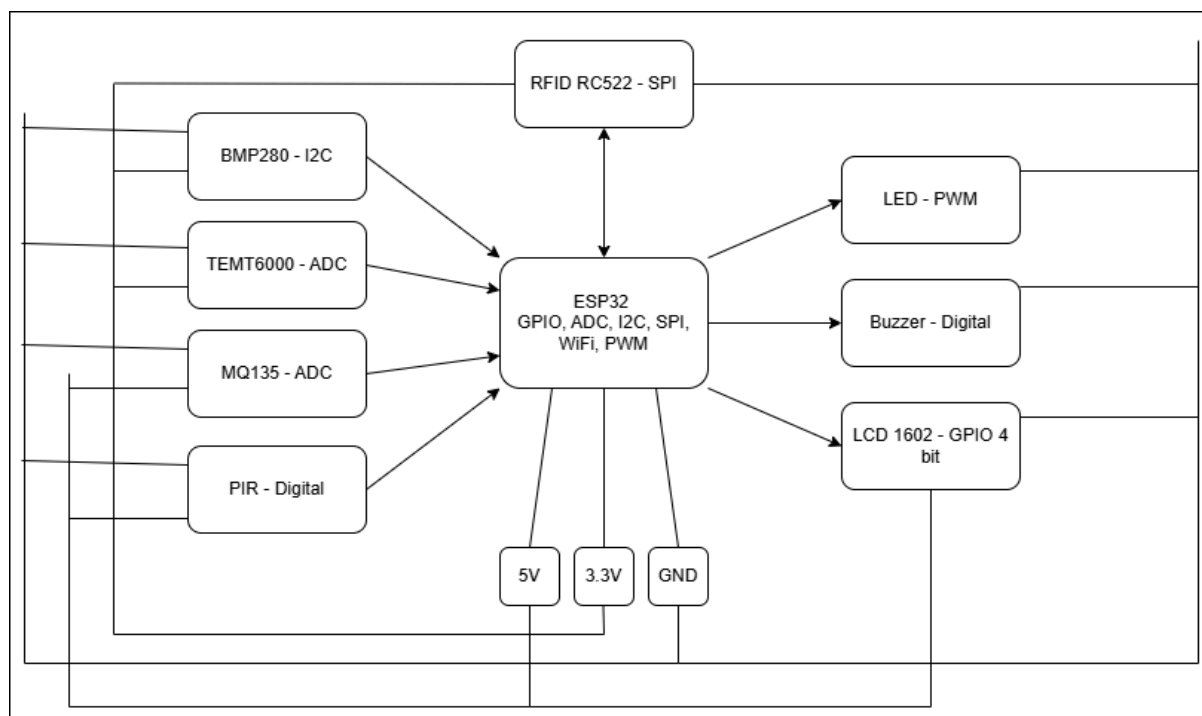


Figura 3.1 Diagrama bloc hardware a sistemului Smart Home

ESP32 este punctul de legătură dintre toate componentele hardware ale prototipului. Acesta citește datele de la senzori, stabilește starea curentă a sistemului și controlează elementele de ieșire, precum LED-ul RGB, buzzerul și afișajul LCD. În același timp, prin conexiunea WiFi, ESP32

comunică cu brokerul MQTT, transmite valorile către aplicația PWA și primește comenzile introduse de utilizator. Din acest motiv, placa ESP32 reprezintă nucleul hardware al proiectului.

Senzorii conectați la ESP32 sunt folosiți atât pentru monitorizarea mediului, cât și pentru funcții de securitate. BMP180 oferă valori pentru temperatură și presiune atmosferică, TEMT6000 indică nivelul de lumină, MQ135 este folosit pentru detectarea unor variații asociate gazului, iar PIR BISS0001 detectează mișcarea. Prin combinarea acestor senzori, sistemul poate urmări mai multe aspecte ale spațiului supravegheat, nu doar o singură valoare izolată.

Elementele de ieșire oferă feedback local și fac vizibilă starea sistemului chiar și fără aplicația PWA. LED-ul RGB poate semnaliza diferite stări, poate fi controlat din aplicație și poate primi valori asociate cardurilor RFID. Buzzerul activ este folosit pentru avertizare sonoră în situații de alarmă sau la citirea cardurilor, iar LCD-ul 1602 afișează mesaje scurte direct pe dispozitiv. Aceste componente completează interfața web prin reacții vizibile în mediul fizic.

Modulul RFID RC522 adaugă o funcție de identificare în arhitectura hardware. Cardurile sunt citite prin acest modul, iar UID-ul obținut este transmis către ESP32 pentru verificare și procesare. În funcție de modul de lucru, un card poate fi adăugat, șters, recunoscut ca valid sau asociat unei configurații. Astfel, utilizatorul poate interacționa cu sistemul și printr-un element fizic, nu doar prin aplicația web.

În ansamblu, structura hardware aleasă este potrivită pentru un prototip Smart Home, deoarece combină module accesibile și suficient de variate pentru a demonstra mai multe scenarii IoT. ESP32 asigură partea de control și comunicație, senzorii furnizează date despre mediu, elementele de ieșire oferă feedback local, iar RFID-ul permite interacțiuni bazate pe identificare. Această structură pregătește secțiunile următoare, în care fiecare componentă este descrisă mai detaliat.

3.2. Modulul ESP32

Modulul ESP32 reprezintă componenta centrală a sistemului hardware, deoarece prin el sunt realizate citirea senzorilor, controlul elementelor conectate și comunicația wireless. În proiectul Smart Home, ESP32 face legătura dintre mediul fizic și aplicația PWA: preia date de la senzori, controlează LED-ul RGB, buzzerul și LCD-ul, gestionează modulul RFID și transmite informațiile către brokerul MQTT prin WiFi.

Alegerea ESP32 este potrivită pentru acest prototip deoarece placa oferă conectivitate WiFi integrată și periferice hardware variate. Documentația Espressif descrie ESP32 ca un cip care include WiFi la 2,4 GHz și Bluetooth, fiind destinat aplicațiilor conectate și sistemelor embedded[RW11]. Pentru proiect, acest lucru reduce numărul de module externe necesare, deoarece placa se poate conecta direct la rețea și la infrastructura MQTT.

Pinii GPIO sunt folosiți pentru componente digitale, precum senzorul PIR, butonul, buzzerul și afișajul LCD. Pentru senzorii analogici, ESP32 folosește intrările ADC, care transformă tensiunile citite în valori numerice.

Periferele oferite de acesta pentru comunicație sunt esențiale în realizarea sistemului propus deoarece fără SPI și I2C comunicarea cu cititorul RFID și senzorul de temperatură și presiune atmosferică ar fi imposibilă.

Prin periferele disponibile și layout-ul de pini, ESP32 oferă suportul necesar pentru funcțiile principale ale prototipului: monitorizare, control local, semnalizare, identificare RFID și comunicare cu aplicația PWA. Din acest motiv, placa este potrivită pentru un sistem Smart Home experimental, în care mai multe componente trebuie integrate într-o arhitectură compactă.

3.3. Senzorii utilizați

Senzorii reprezintă nivelul de intrare al sistemului Smart Home, deoarece prin ei ESP32 primește informații despre mediul monitorizat. Fără aceste componente, placa ar putea executa doar comenzi primite de la utilizator, dar nu ar putea observa schimbări din locuință și nu ar putea reacționa automat. În proiectul de față, senzorii au fost aleși pentru a acoperi atât parametri de confort, cât și situații legate de siguranță: temperatură, presiune atmosferică, lumină, variații asociate gazului și mișcare.

Senzorul BMP180 este folosit pentru măsurarea temperaturii și presiunii atmosferice. Acesta comunică prin I2C, folosind o linie pentru date și una pentru ceas, ceea ce face conectarea la ESP32 mai simplă. Rolul BMP180 este mai ales informativ, deoarece temperatura și presiunea nu declanșează direct o alarmă, dar oferă date utile despre spațiul monitorizat. Documentația Bosch Sensortec descrie BMP180 ca senzor digital de presiune barometrică, utilizabil și pentru determinarea temperaturii necesare compensării măsurării presiunii[RW2].

Senzorul TCM6000 este utilizat pentru estimarea nivelului de lumină ambientală. Acesta funcționează ca fototranzistor și oferă un semnal analogic citit de ESP32 prin ADC. În lucrare, valoarea citită nu este folosită pentru măsurători exacte în lux, ci pentru diferențierea practică dintre zi și noapte. Această abordare este potrivită pentru prototip, deoarece scopul este adaptarea comportamentului sistemului la condițiile generale de lumină. Fișa tehnică Vishay descrie TCM6000X01 ca senzor de lumină ambientală bazat pe fototranzistor într-o capsulă miniaturală[B8].

Senzorul MQ135 este introdus pentru detectarea variațiilor asociate gazelor sau calității aerului. El transmite un semnal analogic, iar ESP32 compară valoarea citită cu pragurile stabilite în firmware. În prototip, MQ135 nu este folosit ca instrument profesional de măsurare, ci ca element de avertizare orientativă. Această precizare este importantă, deoarece senzorii din familia MQ au nevoie

de calibrare și condiții controlate pentru măsurători precise. Manualul Winsen descrie MQ135 ca senzor semiconductor pentru calitatea aerului, bazat pe modificarea conductivității materialului sensibil în funcție de gazele detectate[B9].

Senzorul PIR BISS0001 este folosit pentru detectarea mișcării în spațiul supravegheat. Acesta nu identifică o persoană și nu oferă poziția exactă, ci semnalează o variație detectată prin principiul infraroșu pasiv. Dacă alarma este armată și senzorul indică mișcare, sistemul poate publica un eveniment MQTT și poate activa semnalizarea locală. Documentația BISS0001 prezintă circuitul ca un controler pentru detecție PIR, construit pentru aplicații cu consum redus[B10].

Integrarea acestor senzori permite sistemului să combine monitorizarea ambientală cu funcții de siguranță. BMP180 și TEMT6000 oferă informații despre condițiile generale din spațiu, în timp ce MQ135 și PIR sunt folosiți pentru evenimente care pot cere o reacție din partea sistemului. Această împărțire ajută la organizarea logicii firmware-ului, deoarece datele nu sunt tratate la fel: temperatura și presiunea au rol informativ, lumina poate influența anumite comportamente, iar gazul și mișcarea pot declanșa alarme.

3.4. Actuatori și elemente de semnalizare

Actuatoarele și elementele de semnalizare sunt componentele prin care sistemul Smart Home produce reacții vizibile sau sonore în mediul fizic. Dacă senzorii colectează date, aceste elemente exprimă starea sistemului și execută efectele comenzilor primite din aplicație sau generate automat de firmware. În proiect, principalele componente de ieșire sunt LED-ul RGB, buzzerul activ și afișajul LCD 1602. LED-ul oferă feedback vizual prin culoare, buzzerul transmite avertizări sonore, iar LCD-ul afișează mesaje locale.

LED-ul RGB este folosit pentru semnalizare vizuală și pentru controlul culorii din aplicația PWA. Acesta are trei canale, roșu, verde și albastru, comandate separat de ESP32. Prin modificarea intensității fiecărui canal se pot obține culori diferite, ceea ce permite folosirea LED-ului atât pentru control manual, cât și pentru indicarea unor stări ale sistemului. Utilizatorul poate trimite valori RGB din aplicație, iar ESP32 le aplică prin semnale PWM. Documentația Arduino ESP32 prezintă perifericul LEDC ca mecanism destinat controlului intensității LED-urilor și generării de semnale PWM[B11].

Buzzerul activ este folosit pentru feedback sonor. Într-un sistem Smart Home, avertizarea auditivă este utilă deoarece poate atrage atenția utilizatorului chiar și atunci când acesta nu urmărește aplicația PWA sau afișajul LCD. Spre deosebire de LED, care transmite informația vizual, buzzerul oferă un semnal rapid, potrivit pentru evenimente care necesită atenție imediată. Din punct de vedere hardware, acesta este comandat de ESP32 printr-un pin digital.

Afişajul LCD 1602 este folosit pentru mesaje scurte afişate direct pe dispozitiv. Acesta completează aplicaţia PWA, deoarece permite observarea unor stări fără deschiderea interfeţei web. În proiect, LCD-ul poate afişa mesaje de pornire, starea sistemului, informaţii despre alarmă sau reacţii la comenzi. Biblioteca LiquidCrystal permite controlul afişajelor compatibile cu driverul Hitachi HD44780, în mod de lucru pe 4 sau 8 biţi[RW2]. În implementarea hardware, LCD-ul este conectat direct la ESP32 în mod 4-bit, fără modul I2C intermediar. Această soluţie foloseşte mai mulţi pini GPIO, dar oferă control direct asupra liniilor de date şi comandă. Controlerul HD44780 este destinat afişării de caractere alfanumerice şi simboluri pe LCD-uri de tip matrice de puncte[RW12].

Componentă	Tip control	Conectare la ESP32	Rol în sistem
LED RGB	PWM, prin trei canale separate	R GPIO25, G GPIO26, B GPIO16	Semnalizare vizuală, control culoare din aplicaţie, feedback asociat cardurilor RFID sau alarmelor.
Buzzer activ	Ieşire digitală	GPIO32	Avertizare sonoră pentru alarmă, citire RFID sau evenimente importante.
LCD 1602 HD44780	Control paralel în mod 4-bit	RS GPIO2, EN GPIO15, D4 GPIO13, D5 GPIO12, D6 GPIO14, D7 GPIO27	Afişare locală de mesaje privind starea sistemului şi evenimentele principale.

Tabelul 3.1. Actuatori şi elemente de afişare utilizate în sistem

Aceste componente au fost alese pentru a oferi un feedback local uşor de înţeles. Aplicaţia PWA este interfaţa principală a sistemului, dar semnalizarea locală rămâne importantă atunci când utilizatorul se află lângă dispozitiv sau nu urmăreşte activ browserul. LED-ul, buzzerul şi LCD-ul permit prototipului să afişeze stări direct în mediul fizic, fără să depindă complet de aplicaţie. Astfel, sistemul rămâne autonom la nivel local şi conectat la nivel IoT.

Prin integrarea acestor elemente, sistemul devine mai uşor de testat şi de folosit. Mesajele MQTT şi valorile senzorilor sunt importante în aplicaţie, însă utilizatorul are nevoie şi de confirmări vizibile sau sonore lângă dispozitiv. LED-ul RGB, buzzerul activ şi LCD-ul 1602 îndeplinesc acest rol şi transformă prototipul într-un sistem care nu doar transmite date, ci reacţionează local la stările detectate.

3.5. Modulul RFID RC522

Modulul RFID RC522 este folosit în proiect pentru identificare locală prin carduri sau taguri RFID. Prin această componentă, sistemul Smart Home nu se limitează la senzori și actuatoare, ci permite și interacțiunea cu un obiect fizic. Când utilizatorul apropie un card de cititor, RC522 citește identificatorul acestuia, iar ESP32 îl interpretează în funcție de logica din firmware. Astfel, sistemul poate adăuga un card nou, poate verifica un card existent, poate aplica o configurație asociată sau poate trimite evenimentul către aplicația PWA.

Tehnologia RFID permite identificarea fără contact direct, prin comunicație radio între cititor și tag. În proiect, această funcție este realizată cu modulul RC522, bazat pe circuitul MFRC522. Documentația NXP descrie MFRC522 ca un circuit integrat pentru comunicație contactless la frecvența de 13,56 MHz, cu suport pentru ISO/IEC 14443 A, MIFARE și NTAG[B12]. Din acest motiv, modulul este potrivit pentru un prototip în care UID-ul cardului este citit și folosit pentru acces sau personalizare.

În arhitectura hardware, RC522 comunică direct cu ESP32 prin SPI. Această interfață permite schimbul rapid de date între microcontroler și modulul RFID, folosind linii pentru ceas, date transmise, date recepționate și selecția dispozitivului. În proiect, conexiunile principale sunt SS, SCK, MOSI, MISO și RST, iar ESP32 le folosește pentru inițializarea cititorului și citirea UID-ului. Biblioteca MFRC522 din ecosistemul Arduino este destinată citirii și scrierii cardurilor RFID prin module bazate pe RC522, conectate prin SPI[B13].

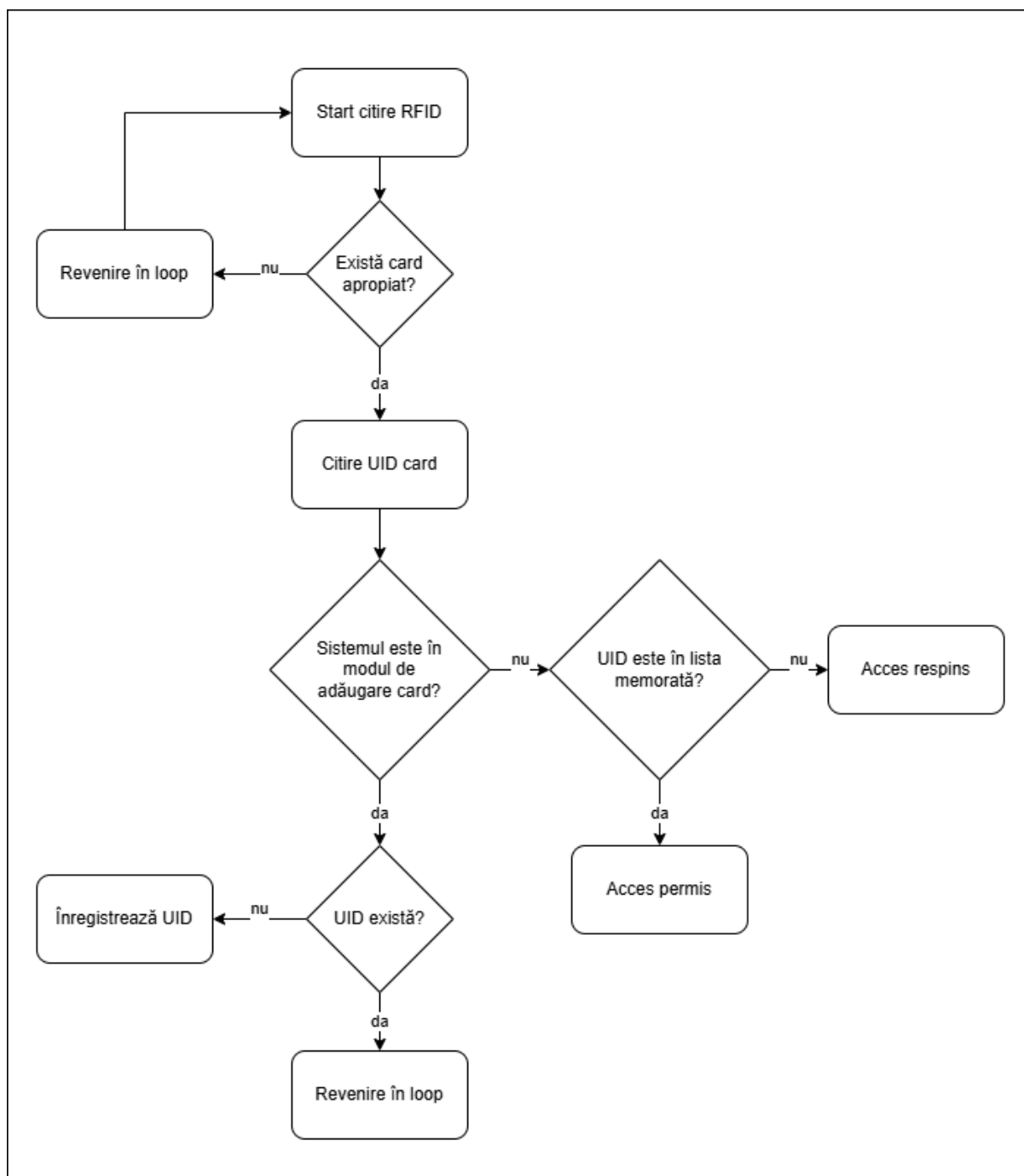


Figura 3.2. Fluxul hardware pentru citirea și tratarea unui card RFID

Rolul RFID-ului în proiect nu este de a realiza un sistem profesional de acces, ci de a demonstra integrarea identificării prin card într-un sistem IoT. UID-ul citit de RC522 este folosit ca identificator, iar firmware-ul verifică dacă acesta există în lista memorată. În funcție de starea sistemului, cardul poate fi adăugat, șters, recunoscut ca valid sau asociat unei configurații, cum ar fi o anumită culoare pentru LED-ul RGB ori activarea feedback-ului sonor.

Prin integrarea modului RFID RC522, sistemul Smart Home primește o formă suplimentară de interacțiune fizică. Utilizatorul poate influența comportamentul sistemului prin apropierea unui

card, iar ESP32 transformă această acțiune într-o decizie software. Legătura dintre RFID, MQTT, memoria locală, LED-ul RGB, buzzer și aplicația PWA arată cum o componentă simplă poate fi integrată într-o arhitectură IoT mai largă.

3.6. Schema de conectare și pinout

Schema de conectare arată modul în care componentele hardware ale sistemului Smart Home sunt legate la placa ESP32. Ea transformă arhitectura bloc într-o configurație fizică exactă, în care fiecare senzor, actuator sau modul are pini alocați și funcții clare. Într-un proiect cu microcontroler, această etapă este importantă deoarece o alegere nepotrivită a pinilor poate afecta atât funcționarea firmware-ului, cât și stabilitatea montajului.

În prototip, ESP32 se află în centrul schemei, iar componentele sunt organizate după tipul semnalului folosit. MQ135 și TEMT6000 sunt conectați la intrări ADC, deoarece transmit valori analogice. BMP180 comunică prin I2C, folosind liniile SDA și SCL, iar modulul RFID RC522 este conectat prin SPI, cu linii pentru ceas, date, selecție și resetare. LED-ul RGB folosește trei ieșiri PWM, buzzerul este comandat printr-un pin digital, iar LCD-ul 1602 este conectat în mod paralel, pe 4 biți, prin mai multe linii GPIO.

O regulă importantă a schemei este existența unei mase comune. Toate modulele conectate la ESP32 trebuie să împartă aceeași referință GND, deoarece semnalele citite sau comandate de microcontroler au nevoie de un reper electric comun. Fără această legătură, citirile analogice pot deveni instabile, iar semnalele digitale pot fi interpretate greșit. De asemenea, alimentarea fiecărei componente trebuie aleasă corect, iar semnalele aplicate intrărilor ESP32 trebuie să rămână compatibile cu logica de 3,3 V.

Pin ESP32	Componentă	Funcție
GPIO34	MQ135	Intrare analogică pentru senzorul de gaz
GPIO35	TEMT6000	Intrare analogică pentru senzorul de lumină

Tabelul 3.2. Alocarea pinilor ESP32 în sistemul Smart Home (1/3)

Pin ESP32	Componentă	Funcție
GPIO21	BMP180	SDA pentru I2C
GPIO22	BMP180	SCL pentru I2C
GPIO36	PIR BISS0001	Intrare digitală pentru detecția mișcării
GPIO5	RFID RC522	SS, selecție dispozitiv SPI
GPIO18	RFID RC522	SCK, ceas SPI
GPIO23	RFID RC522	MOSI, date către modul
GPIO19	RFID RC522	MISO, date către ESP32
GPIO4	RFID RC522	RST
GPIO25	LED RGB	Canal roșu, PWM
GPIO26	LED RGB	Canal verde, PWM
GPIO16	LED RGB	Canal albastru, PWM
GPIO33	Buton fizic	Intrare digitală cu pull-up intern

Tabelul 3.2. Alocarea pinilor ESP32 în sistemul Smart Home (2/3)

Pin ESP32	Componentă	Funcție
GPIO2	LCD 1602	RS
GPIO15	LCD 1602	EN
GPIO13	LCD 1602	D4
GPIO12	LCD 1602	D5
GPIO14	LCD 1602	D6
GPIO27	LCD 1602	D7
GND	LCD 1602	RW
GND	Toate componentele	Masă comună
5 V sau 3,3 V, după modul	Senzori și module	Alimentare

Tabelul 3.2. Alocarea pinilor ESP32 în sistemul Smart Home (3/3)

Trebuie luată în calcul și alimentarea sistemului și unele particularități ale pinilor ESP32, de aceea alegerea unei mase comune și legarea fiecărei componente la alimentarea corectă pentru aceasta fie ea 3.3V fie 5V este esențială când se stabilește pinou-ul.

Prin stabilirea pinout-ului, sistemul devine mai ușor de implementat și de verificat. Fiecare componentă are o poziție clară în schema hardware, iar firmware-ul poate fi explicat prin raportare directă la pinii folosiți. Tabelul de pinout ajută la refacerea montajului, la verificarea conexiunilor și la identificarea rapidă a eventualelor erori de cablare. Schema de conectare face trecerea de la arhitectura generală la implementarea fizică a prototipului Smart Home.

Capitolul 4. Implementarea firmware-ului ESP32

4.1. Mediul de dezvoltare și bibliotecile utilizate

Implementarea firmware-ului pentru sistemul Smart Home a fost realizată în mediul Arduino, folosind limbajul C++ și suportul oferit pentru placa ESP32. Alegerea acestui mediu de dezvoltare este potrivită pentru un proiect de licență cu caracter aplicativ, deoarece permite configurarea rapidă a plăcii, integrarea bibliotecilor necesare și testarea directă a componentelor conectate. Firmware-ul este organizat într-un fișier de tip sketch, iar structura sa respectă modelul obișnuit al programelor Arduino, cu o etapă de inițializare și o buclă principală de execuție.

Mediul Arduino oferă un echilibru util între accesibilitate și control hardware. Pe de o parte, dezvoltatorul poate lucra cu funcții de nivel înalt pentru WiFi, MQTT, I2C, SPI sau afișare LCD. Pe de altă parte, firmware-ul păstrează controlul direct asupra pinilor ESP32, asupra valorilor citite de la senzori și asupra reacțiilor locale ale sistemului. Această combinație este importantă pentru proiect, deoarece sistemul trebuie să integreze simultan senzori analogici, senzori digitali, module de comunicație și elemente de semnalizare.

Firmware-ul folosește mai multe biblioteci, fiecare având un rol clar în arhitectura software. Unele biblioteci asigură comunicația cu exteriorul, precum WiFi și PubSubClient, altele permit comunicarea cu modulele hardware, precum Wire, SPI, MFRC522 și Adafruit BMP085, iar altele sunt folosite pentru organizarea datelor, afișare sau stocare locală, precum ArduinoJson, LiquidCrystal și Preferences. Prin folosirea acestor biblioteci, codul poate rămâne concentrat pe logica sistemului, fără a reimplementa de la zero mecanismele de comunicație sau de acces la componente.

Biblioteca WiFi este folosită pentru conectarea ESP32 la rețeaua wireless. Această conexiune este necesară deoarece dispozitivul trebuie să comunice cu brokerul MQTT și să transmită date către aplicația PWA. După conectarea la rețea, firmware-ul poate inițializa clientul MQTT și poate publica sau recepționa mesaje. În cadrul sistemului, WiFi-ul reprezintă infrastructura de comunicație de bază, iar fără această etapă ESP32 ar funcționa doar ca sistem local, fără integrare cloud.

Pentru comunicația MQTT este utilizată biblioteca PubSubClient. Aceasta permite conectarea la un broker MQTT, publicarea mesajelor pe topicuri și abonarea la topicurile de comandă. Documentația Arduino pentru PubSubClient precizează că biblioteca permite trimiterea și primirea mesajelor MQTT și oferă suport pentru MQTT 3.1.1[RW13]. În proiect, PubSubClient este folosită pentru publicarea valorilor senzorilor, trimiterea evenimentelor de alarmă, transmiterea logurilor și recepționarea comenzilor trimise din aplicația PWA.

Biblioteca Wire este folosită pentru interfața I2C, necesară senzorului BMP180. Acest senzor transmite valorile de temperatură și presiune către ESP32, iar firmware-ul le include în mesajele de senzori publicate prin MQTT. Pentru lucrul cu BMP180 este utilizată biblioteca Adafruit BMP085,

compatibilă cu senzorii BMP085 și BMP180[RW14]. Prin această bibliotecă, citirea datelor atmosferice se face mai simplu, iar codul firmware-ului poate folosi funcții dedicate pentru obținerea temperaturii și presiunii.

Biblioteca SPI este necesară pentru comunicația cu modulul RFID RC522. Acesta folosește o magistrală SPI pentru schimbul de date cu ESP32, iar biblioteca MFRC522 oferă funcții pentru detectarea cardurilor și citirea UID-ului. În arhitectura proiectului, RFID-ul are un rol important deoarece permite adăugarea, ștergerea și configurarea cardurilor. Integrarea bibliotecii MFRC522 reduce complexitatea interacțiunii directe cu registrul modulului și permite concentrarea pe logica de identificare și memorare.

Pentru construirea și interpretarea mesajelor MQTT în format JSON este folosită biblioteca ArduinoJson. Aceasta este potrivită pentru proiecte embedded în care datele trebuie serializate sau deserializate într-o formă ușor de transmis prin rețea. Documentația ArduinoJson 6 include exemple, referință API și explicații privind lucrul cu obiecte JSON în proiecte Arduino[RW15]. În sistemul Smart Home, ArduinoJson este folosită pentru mesajele de senzori, comenzile pentru LED, configurarea cardurilor RFID și transmiterea logurilor.

Biblioteca LiquidCrystal este folosită pentru controlul afișajului LCD 1602 compatibil HD44780. În acest proiect, afișajul este conectat direct la ESP32 în mod 4-bit, iar firmware-ul îl utilizează pentru mesaje locale de stare. Documentația Arduino arată că LiquidCrystal permite controlul afișajelor LCD compatibile cu chipsetul Hitachi HD44780, în mod 4-bit sau 8-bit[RW16]. Această bibliotecă simplifică afișarea mesajelor, deoarece programul poate folosi funcții precum poziționarea cursorului și scrierea textului.

Pentru stocarea persistentă a cardurilor RFID și a configurațiilor asociate este folosită biblioteca Preferences. Aceasta oferă acces la memoria nevolatilă a ESP32, astfel încât anumite date să rămână disponibile după repornirea sistemului. Documentația Arduino ESP32 descrie Preferences ca bibliotecă specifică platformei ESP32, bazată pe zona NVS, unde datele pot fi păstrate între reporniri și pierderi de alimentare[RW12]. În proiect, această funcție este necesară deoarece lista cardurilor RFID și setările lor nu trebuie pierdute după resetarea plăcii.

Prin utilizarea acestor biblioteci, firmware-ul este organizat în jurul unor responsabilități clare. Comunicarea cu brokerul este separată de citirea senzorilor, citirea RFID este separată de afișarea locală, iar construirea mesajelor JSON este separată de stocarea persistentă. Această împărțire ajută la înțelegerea codului și permite explicarea sa pe module logice în secțiunile următoare. Într-un proiect de licență, această claritate este importantă deoarece cititorul trebuie să poată urmări traseul datelor de la senzor până la aplicație și traseul comenzilor de la aplicație până la componentele fizice.

4.2. Inițializarea sistemului

Inițializarea sistemului reprezintă etapa în care firmware-ul pregătește toate resursele necesare pentru funcționarea prototipului Smart Home. Această etapă este realizată în funcția `setup` și se execută o singură dată, imediat după pornirea sau resetarea plăcii ESP32. Rolul ei este de a configura pinii, perifericele, bibliotecile, comunicația locală cu modulele hardware, memoria nevolatilă și conexiunile de rețea. Dacă această etapă nu este realizată corect, bucla principală nu poate gestiona în mod stabil citirea senzorilor, controlul actuatorilor, RFID-ul sau schimbul de mesaje MQTT.

Prima operație efectuată în firmware este inițializarea comunicației seriale. Aceasta este utilă în timpul dezvoltării și testării, deoarece permite afișarea unor mesaje de diagnostic în monitorul serial. După pornirea comunicației seriale, firmware-ul configurează pinii de intrare și ieșire. Senzorul PIR este setat ca intrare, butonul fizic este configurat ca intrare cu rezistență internă de pull-up, iar buzzerul este setat ca ieșire digitală. Buzzerul este trecut inițial în stare inactivă, pentru ca sistemul să nu emită semnale sonore nedorite imediat după pornire. Această etapă stabilește direcția de lucru pentru pinii folosiți în interacțiunile locale și pregătește firmware-ul pentru citirea evenimentelor fizice și pentru semnalizarea sonoră.

Următoarea etapă privește configurarea LED-ului RGB. Pentru cele trei canale de culoare sunt configurate canalele PWM, frecvența și rezoluția de lucru, apoi fiecare canal este atașat pinului corespunzător. După configurarea canalelor, firmware-ul setează LED-ul într-o stare inițială neutră, astfel încât semnalizarea vizuală să pornească de la o valoare controlată.

Afișajul LCD 1602 este inițializat după configurarea LED-ului. Firmware-ul setează dimensiunea afișajului la 16 coloane și 2 rânduri, apoi afișează un mesaj de pornire. După inițializarea LCD-ului, firmware-ul configurează interfața I2C pentru senzorul BMP180. Liniile SDA și SCL sunt stabilite pe pinii alocați în schema hardware, apoi programul încearcă inițializarea senzorului. Dacă BMP180 răspunde corect, sistemul marchează această stare prin mesaj de confirmare. Dacă senzorul nu este detectat, firmware-ul nu oprește complet sistemul, ci afișează un mesaj de avertizare și continuă inițializarea.

Modulul RFID RC522 este inițializat prin interfața SPI. Firmware-ul configurează liniile SCK, MISO, MOSI și SS, apoi pornește modulul prin funcția de inițializare a bibliotecii RFID.

Această etapă pregătește cititorul pentru detectarea cardurilor și pentru citirea UID-urilor. Un alt pas important este inițializarea memoriei nevolatile prin biblioteca Preferences. Firmware-ul deschide spațiul de stocare dedicat sistemului Smart Home și încarcă datele salvate anterior. Aceste date includ lista cardurilor RFID și configurațiile asociate.

După inițializarea componentelor locale, firmware-ul pornește conectarea la rețeaua WiFi. Această etapă este necesară pentru ca ESP32 să poată comunica ulterior cu brokerul MQTT. După stabilirea conexiunii, firmware-ul poate configura clientul MQTT prin setarea adresei brokerului, a portului, a funcției callback și a dimensiunii bufferului pentru mesaje.

În finalul inițializării, sistemul înregistrează un mesaj de boot în logul intern, afișează o stare de pregătire pe LCD și trece către bucla principală. Din acest moment, firmware-ul este pregătit să citească senzorii, să verifice RFID-ul, să proceseze comenzile MQTT și să actualizeze elementele de semnalizare.

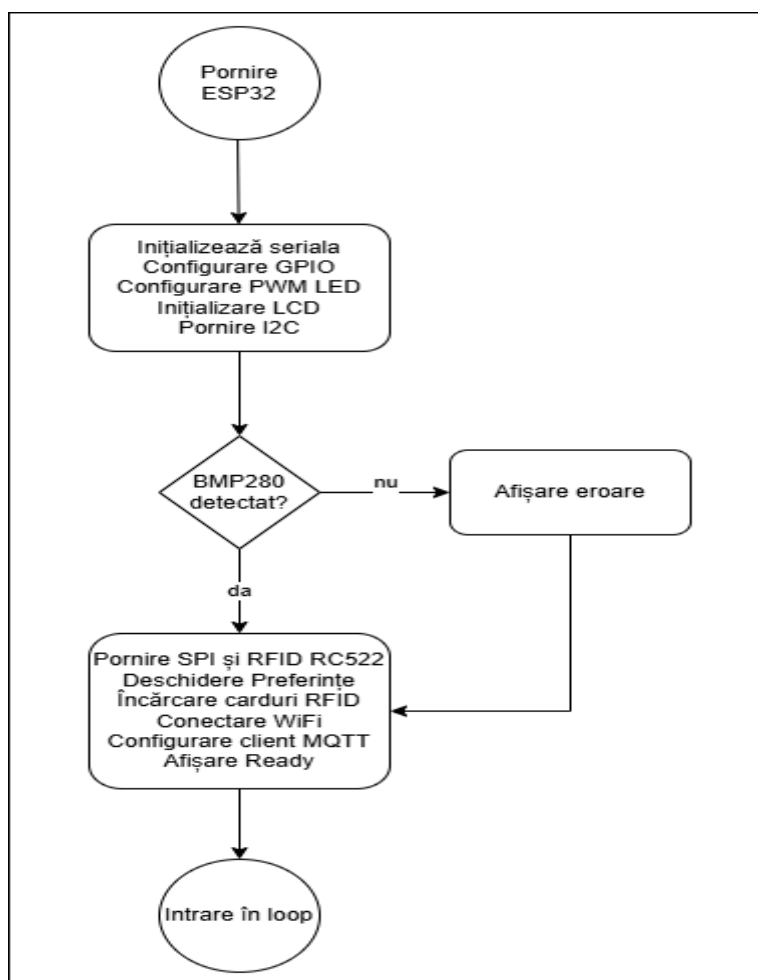


Figura 4.1 Flowchart pentru inițializarea sistemului în funcția setup

Prin această secvență de inițializare, firmware-ul asigură o pornire ordonată a tuturor componentelor. Fiecare modul este pregătit înainte de a fi folosit, iar eventualele probleme sunt

semnalate prin mesaje locale sau seriale. Această etapă oferă baza pentru funcționarea stabilă a sistemului în loop, unde toate componentele inițializate vor fi folosite în mod repetat pentru monitorizare, control, alarmare și comunicație MQTT.

4.3. Bucla principală de execuție

Bucla principală de execuție reprezintă partea firmware-ului care rulează continuu după finalizarea inițializării sistemului. În modelul Arduino, această zonă este implementată în funcția loop, iar rolul ei este de a menține sistemul activ, receptiv și capabil să trateze evenimente venite din mai multe surse. În proiectul Smart Home, loop-ul coordonează comunicația MQTT, citirea periodică a senzorilor, tratarea cardurilor RFID, verificarea butonului fizic, logica de alarmă, controlul LED-ului RGB, funcționarea buzzerului și actualizarea afișajului LCD.

Organizarea buclei principale este importantă deoarece sistemul trebuie să răspundă la evenimente diferite fără să blocheze execuția într-o singură funcție. De exemplu, ESP32 trebuie să poată primi o comandă MQTT în timp ce așteaptă următoarea citire a senzorilor, trebuie să poată detecta un card RFID fără să oprească actualizarea LCD-ului și trebuie să poată controla buzzerul fără să întrerupă verificarea alarmei. Din acest motiv, firmware-ul folosește o structură repetitivă în care fiecare funcție își execută rapid partea de lucru, apoi controlul revine buclei principale.

Primul pas din loop este verificarea conexiunii MQTT. Dacă dispozitivul nu mai este conectat la broker, firmware-ul încearcă reconectarea. Această verificare este plasată la începutul buclei deoarece schimbul de mesaje cu aplicația PWA depinde de conexiunea MQTT. După această etapă, este apelată funcția care permite clientului MQTT să proceseze mesajele primite și să mențină conexiunea activă. Prin această ordine, sistemul rămâne pregătit să primească rapid comenzi din aplicație, precum controlul LED-ului, armarea alarmei sau gestionarea RFID.

După tratarea comunicației MQTT, firmware-ul verifică timpul curent prin mecanismul millis. Această verificare este folosită pentru citirea periodică a senzorilor, fără a introduce întârzieri blocante. Sistemul compară momentul curent cu momentul ultimei publicări, iar dacă intervalul stabilit a fost depășit, actualizează timpul ultimei publicări, citește senzorii și transmite datele prin MQTT. Prin această abordare, senzorii nu sunt citați continuu la fiecare trecere prin loop, ci la intervale controlate, ceea ce reduce încărcarea inutilă a firmware-ului și oferă aplicației valori actualizate într-un ritm stabil.

Citirea senzorilor și publicarea datelor sunt tratate ca două operații consecutive. Mai întâi, firmware-ul actualizează valorile interne ale sistemului: temperatura, presiunea, nivelul de lumină, valoarea senzorului de gaz și starea senzorului PIR. Apoi, aceste valori sunt împachetate într-un mesaj și publicate pe topicul MQTT dedicat senzorilor. Această separare este utilă deoarece datele pot fi

folosite și local, de exemplu pentru alarma de gaz sau pentru stabilirea modului zi/noapte, înainte sau după transmiterea lor către aplicație.

După secțiunea de senzori, loop-ul tratează citirea RFID. Această etapă verifică dacă există un card apropiat de cititor și, în funcție de starea sistemului, poate adăuga cardul, îl poate recunoaște sau poate respinge accesul. Așezarea acestei funcții în bucla principală permite sistemului să detecteze cardurile fără a necesita o comandă separată la fiecare citire. Utilizatorul poate apropia cardul de modulul RFID, iar firmware-ul verifică evenimentul la fiecare trecere prin loop, în paralel cu celelalte funcții ale sistemului.

Următoarea funcție tratează butonul fizic. Acesta oferă o interacțiune locală cu sistemul, independentă de aplicația PWA. Prin verificarea lui în loop, firmware-ul poate detecta apăsarea și poate executa acțiunea asociată, de exemplu modificarea stării alarmei sau o funcție locală de control. Chiar dacă aplicația web este interfața principală a sistemului, existența unui buton fizic este utilă pentru testare și pentru control local.

Logica alarmelor este tratată după verificarea RFID și a butonului. În această etapă, firmware-ul analizează valorile și stările curente pentru a decide dacă alarma de gaz sau alarma de mișcare trebuie activată, menținută sau oprită. Această funcție se bazează pe datele citite de senzori și pe starea internă a sistemului, cum ar fi faptul că alarma este armată sau dezarmată. Prin separarea logicii de alarmă într-o funcție dedicată, codul devine mai ușor de explicat, iar condițiile de declanșare pot fi urmărite mai clar.

Controlul LED-ului RGB este tratat după logica alarmelor, deoarece semnalizarea vizuală poate depinde de priorități. O comandă manuală din aplicație poate seta o culoare, însă o stare de alarmă poate avea prioritate asupra culorii alese manual. În același mod, un card RFID poate avea o culoare asociată, dar sistemul trebuie să decidă dacă acea culoare poate fi afișată în funcție de starea zi/noapte sau de existența unei alarme. Prin această poziționare în loop, LED-ul reflectă starea actualizată a sistemului după ce au fost tratate evenimentele importante.

Buzzerul este gestionat printr-o funcție non-blocking. Aceasta înseamnă că firmware-ul poate produce semnale sonore fără a opri restul execuției. În loc ca programul să rămână blocat într-o secvență de așteptare pentru durata sunetului, funcția verifică timpul curent și schimbă starea buzzerului atunci când este necesar. Această soluție este importantă deoarece sistemul trebuie să rămână receptiv la MQTT, RFID și senzori chiar în timpul unei avertizări sonore.

Ultima funcție apelată în loop este cea care actualizează afișajul LCD. LCD-ul poate prezenta pagini diferite sau mesaje temporare, iar actualizarea lui trebuie făcută controlat, fără a afecta celelalte funcții. Prin tratarea afișajului la finalul buclei, firmware-ul poate prezenta starea rezultată după verificarea conexiunii, citirea senzorilor, tratarea RFID-ului și logica de alarmă. Astfel, mesajele locale reflectă mai bine situația curentă a sistemului.

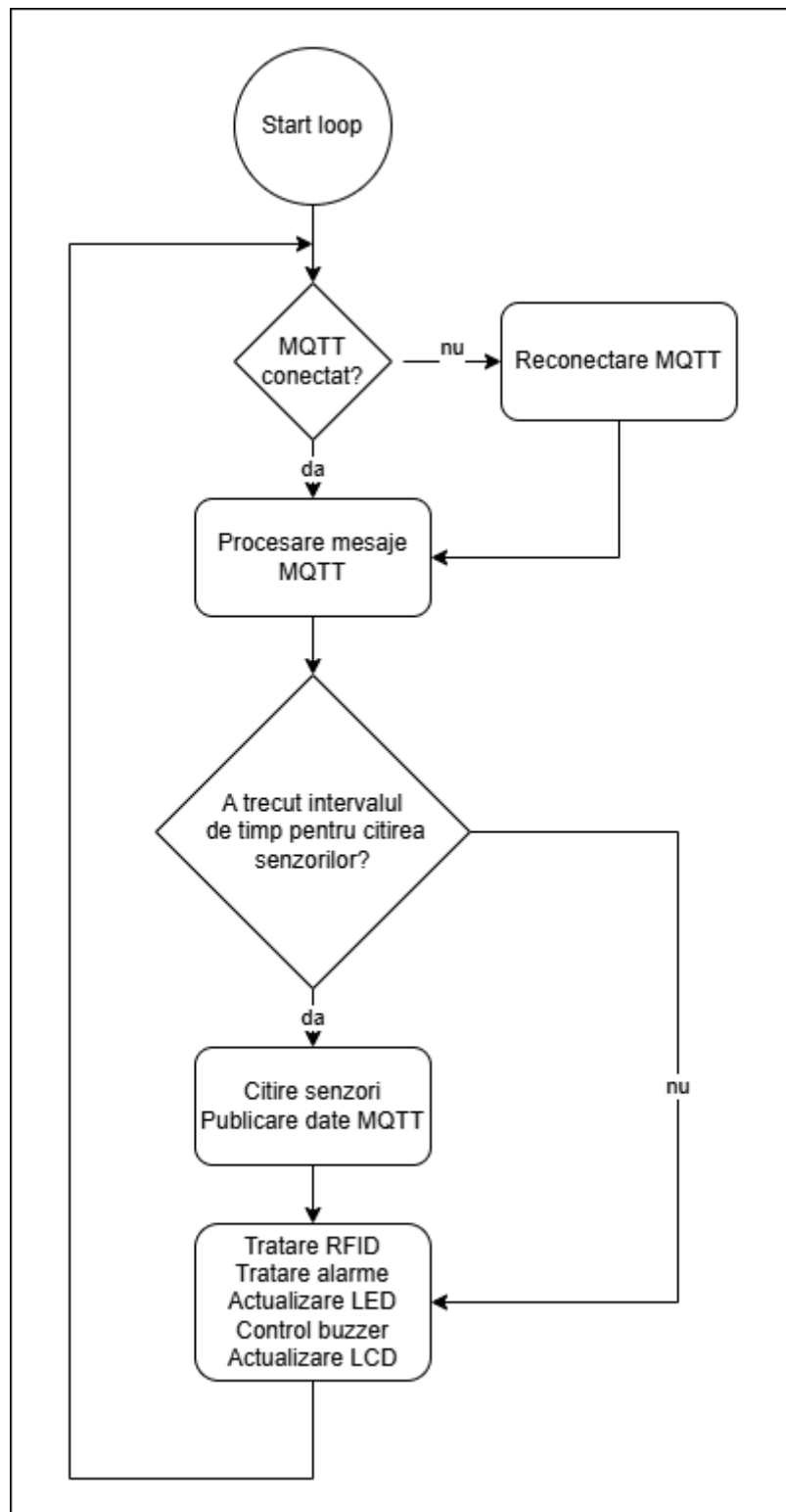


Figura 4.2 Flowchart pentru bucla principală de execuție

Prin această organizare, bucla principală funcționează ca un mecanism de coordonare a întregului firmware. Ea nu conține toată logica sistemului în mod direct, ci apelează funcții specializate, fiecare responsabilă pentru o parte a prototipului. Această structură face programul mai ușor de întreținut și permite explicarea separată a fiecărei funcții în secțiunile următoare. În același

timp, folosirea temporizărilor și a funcțiilor non-blocking permite sistemului să rămână activ, receptiv și capabil să trateze mai multe evenimente într-un mod coerent.

4.4. Citirea senzorilor și publicarea datelor

Citirea senzorilor reprezintă una dintre funcțiile centrale ale firmware-ului, deoarece prin această etapă sistemul obține informațiile necesare pentru monitorizarea locuinței și pentru declanșarea unor reacții automate. În proiectul Smart Home, ESP32 citește periodic temperatura, presiunea atmosferică, nivelul de lumină, valoarea senzorului de gaz și starea senzorului PIR. Aceste date sunt memorate în structura internă a sistemului, apoi sunt folosite atât local, pentru alarme și semnalizare, cât și la distanță, prin publicarea unui mesaj MQTT către aplicația PWA.

Citirea senzorilor este realizată printr-o funcție dedicată, apelată periodic din bucla principală. Intervalul dintre citiri este stabilit printr-o constantă, astfel încât senzorii să nu fie interogați excesiv la fiecare iterație a loop-ului. Această abordare ajută la menținerea unui ritm stabil de actualizare și reduce sarcina inutilă asupra microcontrolerului. În același timp, valorile sunt publicate suficient de des pentru ca utilizatorul să poată urmări starea sistemului în aplicația PWA. În implementarea actuală, intervalul de citire și publicare este de 10 secunde.

Pentru temperatura și presiunea atmosferică, firmware-ul folosește senzorul BMP180. Temperatura este citită direct, iar presiunea este convertită în hectopascali prin împărțirea valorii brute la 100. Aceste valori sunt tratate ca date ambientale și sunt incluse în mesajul general de senzori. Ele nu declanșează direct o alarmă în logica implementată, dar oferă informații utile despre mediul monitorizat și contribuie la caracterul informativ al dashboard-ului.

Pentru citirea temperaturii și presiuni atmosferice se folosește librăria specifică senzorului BMP180, temperatura fiind citită direct, iar presiunea este convertită la hectopascali prin împărțire la 100. Citirile analogice pentru TEMT6000 și MQ135 sunt făcute prin intermediul ADC-ului accesat cu `AnalogRead()` și verificate în funcție de pragul fiecăreia. Senzorul PIR transmite digital starea sa și este citit de firmware prin `DigitalRead()` primind doar valori boolean. Toate aceste date sunt după prelucrate de aplicație.

După citirea și actualizarea stărilor interne, firmware-ul publică datele prin MQTT. Această publicare este realizată într-o funcție separată, care construiește un document JSON cu valorile relevante și îl trimite pe topicul `smarthome/sensors`. Prin această separare, citirea senzorilor și transmiterea datelor rămân două responsabilități distincte. Prima actualizează starea locală a sistemului, iar a doua o transformă într-un mesaj consumat de aplicația PWA.

Mesajul publicat conține atât valori numerice, cât și valori logice. Valorile numerice descriu mediul monitorizat, iar valorile logice descriu stările interne ale sistemului. De exemplu, câmpurile

temp, pres, light și gas sunt folosite pentru afișare directă în dashboard, în timp ce armed, gasAlarm, movAlarm și pir permit aplicației să afișeze starea alarmei și a senzorului de mișcare.

Prin modul în care sunt citite și publicate datele, firmware-ul asigură funcția principală de monitorizare a sistemului Smart Home. Datele sunt obținute local de la senzori, interpretate prin praguri și stări interne, apoi transmise prin MQTT către aplicația PWA. Această secvență arată traseul complet al informației, de la mediul fizic la interfața utilizatorului, și constituie baza funcțiilor de alarmare, afișare și control care vor fi detaliate în secțiunile următoare.

4.5. Comunicarea MQTT în firmware

Comunicarea MQTT reprezintă mecanismul prin care firmware-ul ESP32 schimbă date cu aplicația PWA prin intermediul brokerului configurat pentru sistemul Smart Home. În arhitectura proiectului, ESP32 nu transmite valorile direct către aplicație și nu primește comenzile printr-o conexiune locală directă, ci folosește brokerul MQTT ca punct comun de comunicare. Această soluție permite separarea dintre partea hardware și interfața software, deoarece microcontrolerul publică date pe topicuri cunoscute, iar aplicația PWA le recepționează prin abonare la aceleași topicuri. În sens invers, aplicația publică mesaje de comandă, iar ESP32 le tratează în firmware prin funcția de callback.

În firmware, comunicația MQTT este construită peste conexiunea WiFi a plăcii ESP32. Înainte ca dispozitivul să poată publica mesaje sau să se aboneze la topicuri, sistemul trebuie să se conecteze la rețeaua wireless și să obțină acces la broker. Datele necesare pentru conectare sunt definite în zona de configurare a programului și includ adresa brokerului, portul, utilizatorul, parola și identificatorul clientului MQTT. Într-o variantă de test local, aceste valori pot indica un broker aflat în rețeaua internă, iar pentru integrarea cloud ele se înlocuiesc cu datele oferite de platforma HiveMQ Cloud. Astfel, firmware-ul păstrează aceeași logică MQTT, chiar dacă mediul de rulare al brokerului se modifică.

După conectarea la WiFi, firmware-ul configurează clientul MQTT prin stabilirea serverului, a portului și a funcției care va primi mesajele sosite pe topicurile de comandă. Această funcție de callback este esențială, deoarece prin ea comenzile trimise din aplicația PWA sunt transformate în acțiuni locale. În momentul în care un mesaj ajunge la ESP32, firmware-ul identifică topicul, citește payload-ul, încearcă interpretarea lui ca structură JSON atunci când este cazul și execută operația asociată. Prin această organizare, același mecanism MQTT poate trata funcții diferite, precum controlul LED-ului, armarea alarmei, ștergerea unui card RFID sau trimiterea logurilor.

Reconectarea MQTT este tratată explicit în firmware. Dacă ESP32 pierde conexiunea cu brokerul, sistemul încearcă reconectarea și, după restabilirea legăturii, se abonează din nou la topicurile de comandă. Această etapă este necesară deoarece o deconectare poate apărea din cauza

rețelei WiFi, a brokerului sau a unei reporniri temporare a infrastructurii. Prin abonarea repetată după fiecare reconectare, ESP32 își recuperează capacitatea de a primi comenzi de la aplicația PWA. În același timp, firmware-ul publică un mesaj de status care indică faptul că dispozitivul este online, împreună cu informații precum adresa IP și numărul de carduri RFID memorate.

Topicurile la care ESP32 se abonează sunt cele prin care aplicația PWA poate modifica starea sistemului. Aceste topicuri sunt grupate logic sub prefixul smarthome/cmd, ceea ce face diferența dintre mesajele de comandă și mesajele de stare publicate de microcontroler. Astfel, firmware-ul ascultă comenzile pentru LED, afișaj LCD, adăugare RFID, ștergere RFID, armare alarmă, solicitare loguri și configurare card. Această structură ajută la claritatea comunicației, deoarece fiecare comandă are un canal propriu și poate fi tratată separat în callback.

Topic MQTT	Payload așteptat	Funcție declanșată în firmware	Rezultat produs
smarthome/cmd/light	JSON cu r, g, b sau on	Tratarea comenzii de iluminare	LED-ul RGB primește o culoare nouă sau este stins.
smarthome/cmd/screen	JSON cu line1, line2, duration	Afișarea unui mesaj temporar pe LCD	LCD-ul prezintă un mesaj primit din aplicație sau dintr-un client MQTT extern.
smarthome/cmd/rfid/add	Comandă simplă	Activarea modului de adăugare card	Sistemul așteaptă apropierea unui card RFID pentru memorare.
smarthome/cmd/rfid/delete	JSON cu uid	Ștergerea cardului din lista memorată	Cardul indicat prin UID este eliminat din memoria sistemului.

Tabelul 4.1. Comenzi MQTT tratate de ESP32 (1/2)

Topic MQTT	Payload așteptat	Funcție declanșată în firmware	Rezultat produs
smarthome/cmd/alarm	JSON cu armed	Modificarea stării alarmei	Alarma este armată sau dezarmată, iar stările active sunt resetate la dezarmare.
smarthome/cmd/log	Text simplu request	Trimiterea jurnalului de evenimente	ESP32 publică logurile memorate pe topicul dedicat.
smarthome/cmd/card/config	JSON cu uid, r, g, b, ledOnDay, ledOnNight, buzzerOnRead	Salvarea configurației pentru card	Cardul primește culoare și opțiuni proprii de semnalizare.

Tabelul 4.1. Comenzi MQTT tratate de ESP32 (2/2)

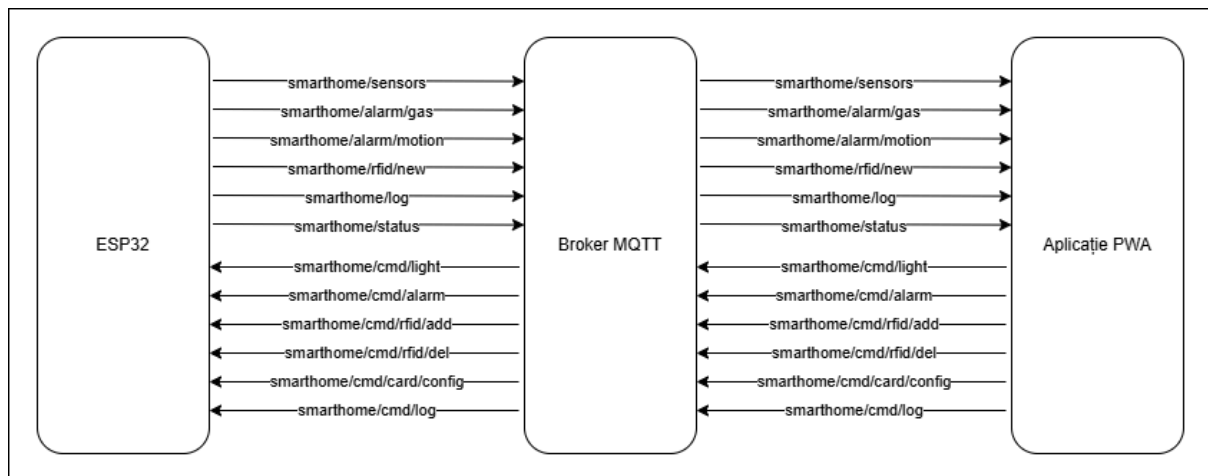


Figura 4.3 Fluxul MQTT dintre ESP32, broker și aplicația PWA

Trimiterea comenzilor are în mare același format pentru toate comenzile aplicația punând în contextul corect comanda pe baza topicului pe care aceasta a venit și verifică payload-ul pentru fiecare în parte din punct de vedere al unei structuri corecte, dar și a datelor pe care urmează să le folosească pentru configurări, actualizări de stări sau acțiuni viitoare și pregătește sistemul să reacționeze corect.

Trimiterea logurilor folosește un mecanism diferit față de comenzile care modifică direct starea sistemului. Atunci când aplicația solicită jurnalul, firmware-ul parcurge intrările memorate și le publică pe topicul smarthome/log, de regulă în loturi, pentru a evita construirea unui mesaj prea mare. Fiecare intrare include un timp relativ și un text scurt, ceea ce permite aplicației să afișeze evenimente precum conectarea MQTT, armarea alarmei, citirea unui card sau declanșarea unei alarme. Această funcție are valoare practică în testare, deoarece oferă o urmă a acțiunilor executate de sistem.

Pe lângă topicurile de comandă, firmware-ul publică mesaje de stare și evenimente. Cel mai important este topicul smarthome/sensors, pe care ESP32 transmite periodic valorile senzorilor și stările interne relevante. Pentru evenimente punctuale, firmware-ul folosește topicuri separate, precum smarthome/alarm/gas și smarthome/alarm/motion. În cazul RFID-ului, adăugarea unui card nou este transmisă pe smarthome/rfid/new, iar mesajele generale de stare pot fi publicate pe smarthome/status. Această separare permite aplicației PWA să trateze diferit datele periodice, alarmele, evenimentele RFID și logurile.

Un aspect important în implementarea MQTT este controlul dimensiunii mesajelor. Firmware-ul folosește documente JSON cu dimensiuni prestabilite, astfel încât memoria să fie gestionată previzibil. Pentru mesajele de senzori este suficient un payload mai redus, în timp ce pentru loguri sau configurări este necesar un spațiu mai mare. Această alegere este potrivită pentru un sistem embedded, deoarece memoria disponibilă nu trebuie consumată necontrolat. În același timp, mesajele trebuie păstrate suficient de compacte pentru a fi transmise rapid și pentru a fi interpretate fără întârzieri vizibile.

Validarea payload-urilor este un alt element necesar în comunicarea MQTT. Firmware-ul trebuie să verifice dacă mesajul primit conține câmpurile necesare înainte de a executa o comandă. De exemplu, ștergerea unui card necesită un UID, configurarea unui card necesită atât UID-ul, cât și valorile de comportament, iar controlul LED-ului are nevoie de valori numerice valide sau de o instrucțiune explicită de oprire. Această verificare reduce riscul ca sistemul să reacționeze incorect la mesaje incomplete, greșite sau trimise pe un topic nepotrivit.

Prin modul în care este implementată, comunicația MQTT oferă firmware-ului o interfață clară cu aplicația PWA. ESP32 publică periodic datele pe care le produce, publică evenimente atunci când starea sistemului se schimbă și ascultă comenzile care pot modifica funcționarea componentelor locale. Brokerul nu conține logica sistemului, ci doar distribuie mesajele, iar această separare face arhitectura mai ușor de urmărit. În ansamblu, comunicația MQTT este elementul care transformă prototipul dintr-un montaj local într-un sistem Smart Home conectat, controlabil printr-o interfață web.

4.6. Logica de alarmă

Logica de alarmă reprezintă una dintre cele mai importante componente ale firmware-ului, deoarece transformă valorile citite de senzori în reacții locale și în evenimente transmise către aplicația PWA. În cadrul sistemului Smart Home implementat, alarma nu este tratată ca un simplu indicator vizual, ci ca o stare internă care influențează comportamentul LED-ului RGB, al buzzerului, al mesajelor afișate local și al mesajelor publicate prin MQTT. Prin această abordare, sistemul poate reacționa atât la valori ridicate ale senzorului de gaz, cât și la detectarea mișcării într-un spațiu supravegheat.

Firmware-ul distinge între două categorii principale de alarmă: alarma de gaz și alarma de mișcare. Alarma de gaz este legată de valoarea analogică furnizată de senzorul MQ135, iar alarma de mișcare este legată de senzorul PIR și de starea de armare a sistemului. Această separare este necesară deoarece cele două evenimente au comportamente diferite. O valoare ridicată a senzorului de gaz trebuie semnalizată indiferent de faptul că alarma generală este armată sau dezarmată, în timp ce mișcarea devine eveniment de alarmă numai atunci când utilizatorul a armat sistemul.

Alarma de gaz funcționează prin compararea valorii citite de la MQ135 cu pragurile definite în firmware. Atunci când valoarea depășește pragul de declanșare, sistemul marchează intern existența unei alarme de gaz, activează semnalizarea locală și publică un mesaj MQTT pe topicul dedicat. Mesajul transmis către aplicație conține informația că evenimentul de gaz a fost declanșat și include valoarea citită de senzor. În aplicația PWA, acest mesaj poate produce afișarea unui banner de avertizare, modificarea stării din dashboard și adăugarea unui eveniment în logul local al interfeței.

Pentru revenirea din alarma de gaz, firmware-ul nu folosește același prag ca la declanșare, ci un prag mai mic. Această diferență între pragul de activare și pragul de revenire evită situația în care alarma se pornește și se oprește rapid atunci când valoarea senzorului oscilează în jurul aceleiași limite. În practică, această tehnică oferă sistemului un comportament mai stabil, deoarece revenirea la starea normală se produce doar atunci când valoarea coboară suficient. După revenire, firmware-ul marchează alarma de gaz ca inactivă, poate opri semnalizarea sonoră și publică un mesaj MQTT prin care aplicația este informată că evenimentul s-a încheiat.

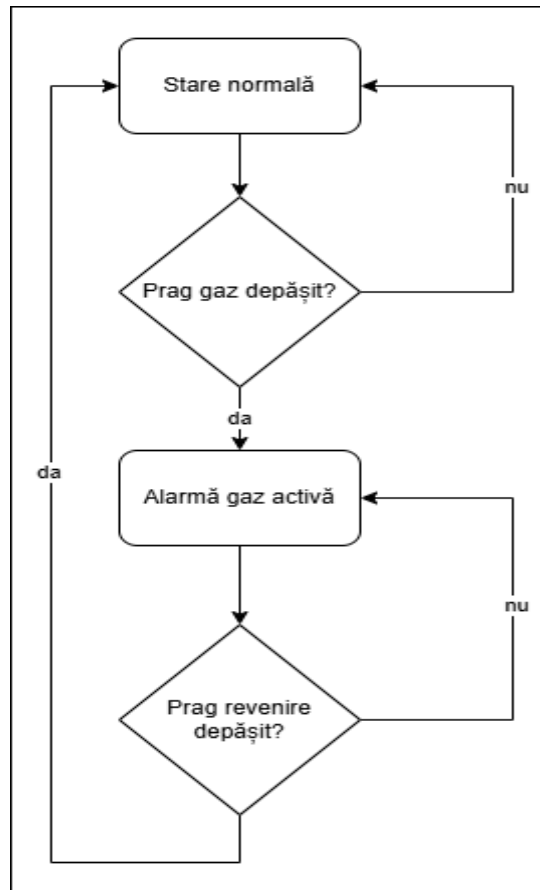


Figura 4.4 Diagrama de stare a alarmei de gaz

Alarma de mișcare este tratată diferit, deoarece depinde de starea de armare a sistemului. Senzorul PIR poate detecta mișcare și atunci când sistemul este dezarmat, dar în acest caz mișcarea este doar o informație de stare, nu un eveniment care trebuie să declanșeze alarma. Dacă utilizatorul armează sistemul din aplicația PWA sau printr-o interacțiune locală, firmware-ul începe să trateze semnalul PIR ca sursă pentru un eveniment de securitate. Astfel, aceeași valoare digitală citită de la senzor capătă semnificații diferite în funcție de starea internă a sistemului.

Pentru mișcare, firmware-ul folosește și o perioadă silențioasă de 60 de secunde. Atunci când senzorul PIR indică o mișcare nouă și alarma este armată, sistemul nu activează imediat alarma sonoră completă, ci intră mai întâi într-o stare de avertizare. Această stare este memorată prin variabilele interne ale sistemului și este transmisă aplicației printr-un mesaj MQTT care indică faptul că mișcarea a fost detectată, dar alarma se află încă în faza silențioasă. Această etapă permite diferențierea dintre detectarea inițială și activarea completă a alarmei.

Dacă perioada silențioasă expiră și sistemul nu a fost dezarmat între timp, firmware-ul trece alarma de mișcare în starea activă. În această stare, semnalizarea locală devine mai evidentă, buzzerul poate fi activat, LED-ul poate reflecta starea de alarmă, iar aplicația primește un eveniment corespunzător. Prin această logică, sistemul oferă un comportament mai realist decât o alarmă

declanșată instantaneu, deoarece există o tranziție între detectarea mișcării și activarea completă. În cazul în care utilizatorul dezarmează sistemul înainte de expirarea perioadei silențioase, firmware-ul anulează starea de alarmă de mișcare și oprește semnalizarea asociată.

Starea de armare este controlată prin mesaj MQTT, pe topicul de comandă al alarmei. Atunci când aplicația PWA transmite un payload cu valoarea armed, firmware-ul modifică starea internă a sistemului. Dacă valoarea primită indică armarea, sistemul intră într-un regim în care mișcarea detectată poate declanșa alarma. Dacă valoarea indică dezarmarea, firmware-ul oprește stările de alarmă de mișcare, dezactivează perioada silențioasă și oprește buzzerul. Această acțiune de resetare este necesară pentru ca sistemul să nu rămână într-o stare de avertizare după ce utilizatorul a decis dezarmarea.

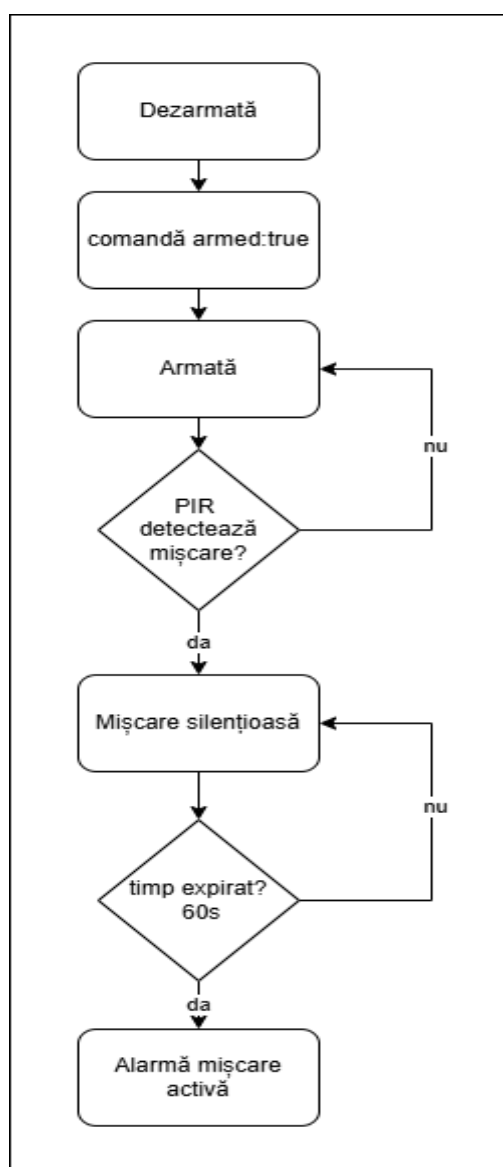


Figura 4.5 Diagrama de stare a alarmei de mișcare

În implementare, starea alarmei este integrată în structura generală a sistemului. Această structură reține dacă alarma este armată, dacă alarma de gaz este activă, dacă alarma de mișcare se află în perioada silențioasă sau dacă a devenit activă. Prin păstrarea acestor valori într-un singur loc, firmware-ul poate decide mai ușor ce trebuie afișat pe LCD, ce culoare trebuie aplicată LED-ului și ce informații trebuie publicate prin MQTT. În plus, aplicația PWA poate primi aceste stări în payload-ul periodic al senzorilor și le poate afișa în dashboard.

Logica de alarmă influențează și prioritatea semnalizării vizuale. LED-ul RGB poate fi controlat manual din aplicație sau poate primi o culoare asociată unui card RFID, însă o stare de alarmă trebuie să aibă prioritate asupra acestor comportamente obișnuite. Din acest motiv, funcția care actualizează LED-ul trebuie să țină cont de stările de alarmă înainte de a aplica o culoare manuală sau o configurație RFID. Această prioritate face ca utilizatorul să observe rapid o stare critică și reduce riscul ca o alarmă să fie ascunsă de o setare decorativă.

Buzzerul are un rol asemănător, dar în plan sonor. El poate fi folosit pentru feedback scurt la citirea unui card sau pentru avertizare în situații de alarmă, însă firmware-ul trebuie să evite blocarea execuției prin semnale sonore lungi tratate cu întârzieri. De aceea, controlul buzzerului este realizat printr-o logică non-blocking, bazată pe temporizări. Această soluție permite sistemului să continue să primească mesaje MQTT, să citească RFID-ul și să actualizeze senzorii chiar în timp ce semnalizarea sonoră este activă.

4.7. Managementul RFID și stocarea cardurilor

Managementul RFID reprezintă partea firmware-ului prin care sistemul Smart Home poate identifica un card apropiat de cititor, poate memora carduri noi, poate elimina carduri existente și poate aplica anumite configurații asociate fiecărui UID. Această funcție extinde sistemul dincolo de monitorizarea senzorilor, deoarece introduce o formă de interacțiune fizică între utilizator și prototip. Prin apropierea unui card de cititorul RC522, utilizatorul poate declanșa un flux de verificare locală, iar ESP32 decide ce acțiune trebuie realizată în funcție de starea curentă a sistemului și de informațiile păstrate în memorie.

În firmware, fiecare card RFID este identificat prin UID. UID-ul citit de modulul RC522 este transformat într-un șir de caractere, într-un format uniform, astfel încât să poată fi comparat cu UID-urile deja memorate. Această normalizare este importantă deoarece același card trebuie recunoscut în mod constant la citiri succesive, fără diferențe produse de litere mici, litere mari sau formatare. După citire, firmware-ul caută UID-ul în lista internă de carduri și stabilește dacă acel card este cunoscut sau necunoscut.

Lista de carduri este organizată într-un vector cu dimensiune fixă, iar numărul maxim de carduri este stabilit printr-o constantă. Această soluție este potrivită pentru un prototip, deoarece limitează consumul de memorie și face comportamentul sistemului previzibil. Fiecare element din

listă nu reține doar UID-ul, ci și o serie de parametri care descriu comportamentul asociat cardului. Astfel, cardul nu este tratat doar ca o cheie simplă de acces, ci ca un obiect de configurare care poate influența culoarea LED-ului RGB și feedback-ul sonor la citire.

Stocarea persistentă a cardurilor este realizată prin biblioteca Preferences, care permite păstrarea datelor în memoria nevolatilă a ESP32. Această funcție este necesară deoarece lista cardurilor nu trebuie pierdută după resetarea plăcii sau după întreruperea alimentării. La pornirea sistemului, firmware-ul încarcă datele salvate anterior, iar în timpul funcționării salvează din nou lista atunci când un card este adăugat, șters sau configurat. Prin această abordare, sistemul păstrează continuitatea între sesiuni și se comportă mai aproape de o aplicație reală de control.

Adăugarea unui card RFID pornește din aplicația PWA, printr-o comandă MQTT publicată pe topicul dedicat. După primirea comenzii, ESP32 activează o stare temporară de așteptare, în care utilizatorul trebuie să apropie cardul de cititor într-un interval limitat. Această stare este utilă deoarece sistemul poate diferenția între o citire normală și o citire destinată înregistrării unui card nou. Dacă un card este detectat în perioada de așteptare, firmware-ul verifică dacă UID-ul există deja în listă și, dacă nu există spațiu sau cardul este duplicat, tratează situația corespunzător. Dacă adăugarea este validă, UID-ul este memorat, lista este salvată în memoria nevolatilă și aplicația este informată printr-un mesaj MQTT.

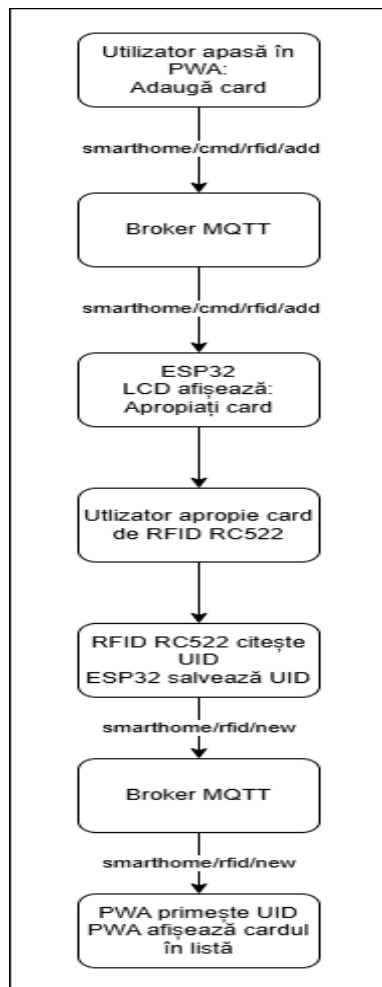


Figura 4.6 Diagrama de secvență pentru adăugarea unui card RFID

Ștergerea unui card se realizează printr-o comandă MQTT care conține UID-ul cardului ce trebuie eliminat. După primirea comenzii, firmware-ul normalizează UID-ul și îl caută în lista cardurilor memorate. Dacă îl găsește, elimină intrarea respectivă și reface lista astfel încât spațiul rămas să fie gestionat corect. O soluție eficientă pentru un vector cu dimensiune fixă este înlocuirea cardului șters cu ultimul card valid din listă, apoi scăderea numărului total de carduri. După această operație, datele sunt salvate din nou în memoria nevolatilă, iar aplicația primește un mesaj de status privind ștergerea.

Configurarea cardurilor RFID oferă sistemului o funcție suplimentară de personalizare. Prin aplicația PWA, utilizatorul poate introduce UID-ul cardului și poate seta valorile pentru culoarea LED-ului RGB, precum și opțiuni privind aprinderea LED-ului ziua, aprinderea LED-ului noaptea și activarea buzzerului la citirea cardului. Firmware-ul primește aceste valori într-un payload JSON, caută cardul în listă și actualizează câmpurile asociate. După salvare, configurația devine persistentă și poate fi aplicată la următoarea citire a cardului.

În funcționarea normală, citirea unui card produce o verificare rapidă în lista cardurilor memorate. Dacă UID-ul este găsit, firmware-ul poate selecta cardul curent și poate aplica setările asociate. De exemplu, dacă sistemul se află în modul noapte și cardul permite aprinderea LED-ului noaptea, culoarea salvată pentru acel card poate fi aplicată LED-ului RGB. Dacă opțiunea `buzzerOnRead` este activă, sistemul poate produce și un semnal sonor scurt. În același timp, evenimentul poate fi înregistrat în log și transmis către aplicație prin MQTT.

Dacă un card necunoscut este citit, sistemul poate trata evenimentul ca acces refuzat sau ca simplă citire neautorizată, în funcție de starea curentă. În acest caz, UID-ul nu este adăugat automat, deoarece adăugarea trebuie să fie inițiată explicit din aplicația PWA. Această separare între citire normală și adăugare este importantă deoarece previne memorarea accidentală a oricărui card apropiat de cititor. Astfel, utilizatorul controlează momentul în care sistemul acceptă înregistrarea unui nou card.

4.8. Controlul LED RGB, LCD și buzzer

Controlul elementelor de ieșire are rolul de a transforma stările interne ale firmware-ului în semnale vizibile sau sonore pentru utilizator. În proiectul Smart Home, aceste elemente sunt LED-ul RGB, afișajul LCD 1602 și buzzerul activ. Ele nu funcționează izolat, ci sunt coordonate de firmware în funcție de alarme, comenzi MQTT, citiri RFID, valori de senzori și starea generală a sistemului. Prin această organizare, ESP32 poate comunica local cu utilizatorul, chiar și atunci când aplicația PWA nu este urmărită în mod activ.

LED-ul RGB este folosit ca element de semnalizare vizuală și ca actuator controlabil din aplicație. Firmware-ul comandă separat cele trei canale de culoare, roșu, verde și albastru, prin semnale PWM. Această comandă permite atât afișarea unei culori alese manual de utilizator, cât și semnalizarea unor stări interne, precum alarma de gaz sau alarma de mișcare. Pentru ca LED-ul să reflecte corect situația sistemului, firmware-ul nu aplică pur și simplu ultima culoare primită, ci folosește o logică de prioritate. Prima prioritate a LED-ului este alarma de gaz, iar a doua prioritate este alarma de mișcare activă.

Perioada silențioasă a alarmei de mișcare nu modifică agresiv LED-ul. În această etapă, firmware-ul păstrează LED-ul în regimul normal sau în regimul stabilit anterior, deoarece alarma nu a trecut încă în starea activă. După tratarea alarmelor, firmware-ul verifică dacă există o comandă manuală pentru LED primită prin MQTT. Dacă utilizatorul a ales o culoare din aplicația PWA, valorile RGB sunt salvate în starea sistemului și sunt aplicate LED-ului. Dacă nu există o alarmă activă și nu există o comandă manuală prioritară, firmware-ul verifică dacă este selectat un card RFID valid. În acest caz, poate aplica setările asociate cardului respectiv.

În lipsa alarmelor, a comenzilor manuale și a unui card activ, firmware-ul aplică un comportament implicit. Acest comportament ține cont de valoarea senzorului de lumină și de starea isDay. Dacă sistemul interpretează mediul ca fiind zi, LED-ul poate rămâne stins, iar dacă mediul este interpretat ca fiind noapte, LED-ul poate fi aprins într-o culoare albă sau într-o stare vizuală implicită. Această regulă oferă sistemului un comportament autonom simplu, bazat pe condițiile de lumină ambientală.

Afișajul LCD 1602 oferă feedback local sub formă de text. Firmware-ul folosește două tipuri principale de afișare: mesaje temporare și rotație normală de pagini. Mesajele temporare apar atunci când sistemul trebuie să comunice imediat o stare, cum ar fi conectarea WiFi, adăugarea unui card, ștergerea unui card, armarea alarmei sau o comandă primită pentru afișaj. Aceste mesaje au o durată stabilită, după care sistemul revine la afișarea normală. Prin acest mecanism, LCD-ul poate prezenta evenimente scurte fără să blocheze definitiv afișarea valorilor de stare.

În situațiile de alarmă, LCD-ul schimbă regimul normal de afișare și prezintă mesaje prioritare. Pentru alarma de gaz, afișajul poate alterna între un mesaj de avertizare și valoarea ADC a senzorului de gaz. Pentru alarma de mișcare activă, LCD-ul poate alterna mesaje care indică detectarea intruziunii și activarea semnalizării. Pentru perioada silențioasă a alarmei de mișcare, afișajul poate prezenta un countdown până la activarea completă a alarmei. Această organizare face ca LCD-ul să reflecte aceeași logică de prioritate ca LED-ul și buzzerul.

Buzzerul activ este folosit pentru feedback sonor și are două moduri principale de utilizare. Primul mod este avertizarea continuă sau intermitentă asociată alarmelor. În acest caz, firmware-ul pornește buzzerul cu o perioadă stabilită, iar funcția de control non-blocking schimbă starea pinului la intervale succesive. Pentru alarma de gaz și pentru alarma de mișcare pot fi folosite perioade diferite, ceea ce permite diferențierea auditivă a evenimentelor.

Al doilea mod de utilizare este feedback-ul scurt, folosit la acțiuni precum citirea unui card RFID sau apăsarea butonului fizic. În aceste situații, buzzerul poate produce unul sau mai multe semnale scurte, care confirmă local faptul că sistemul a detectat interacțiunea. Acest tip de feedback este util deoarece utilizatorul primește imediat o confirmare sonoră, fără să verifice aplicația PWA sau LCD-ul. În același timp, firmware-ul trebuie să păstreze aceste semnale scurte, pentru ca ele să nu afecteze semnificativ celelalte procese.

Prin integrarea acestor trei mecanisme de ieșire, firmware-ul asigură o interacțiune locală completă. LED-ul indică stări și culori, LCD-ul explică prin text ce se întâmplă, iar buzzerul atrage atenția în situații importante. Împreună, ele completează comunicația MQTT și aplicația PWA, astfel încât sistemul Smart Home să poată fi folosit și observat atât la distanță, cât și direct lângă montajul hardware.

4.9. Jurnalizarea evenimentelor

Jurnalizarea evenimentelor are rolul de a păstra o urmă internă a acțiunilor și stărilor importante produse în timpul funcționării sistemului Smart Home. Într-un prototip care combină senzori, alarme, RFID, comenzi MQTT și feedback local, logul devine util atât pentru utilizator, cât și pentru etapa de testare. Prin log, se poate observa dacă sistemul s-a conectat la broker, dacă a primit o comandă, dacă un card a fost citit, dacă alarma a fost armată sau dacă un eveniment de securitate a fost declanșat. Astfel, jurnalizarea completează afișarea în timp real și oferă o formă simplă de diagnostic.

Firmware-ul implementează jurnalizarea printr-un buffer circular. Această soluție este potrivită pentru ESP32 deoarece limitează numărul de evenimente păstrate și evită creșterea necontrolată a memoriei utilizate. În proiect, logul are un număr maxim de intrări, iar fiecare intrare conține timpul relativ al evenimentului și un mesaj text scurt. Atunci când bufferul se umple, evenimentele noi înlocuiesc treptat intrările cele mai vechi. Prin această organizare, sistemul păstrează evenimentele recente, care sunt de obicei cele mai relevante pentru observarea comportamentului curent.

Fiecare intrare de log include un marcaj temporal bazat pe timpul scurs de la pornirea plăcii. Această valoare nu reprezintă ora calendaristică reală, ci timpul intern furnizat de funcția de temporizare a microcontrolerului. Pentru un prototip, această abordare este suficientă deoarece permite ordonarea evenimentelor și observarea succesiunii lor. De exemplu, în timpul testării se poate vedea dacă o comandă MQTT a fost urmată de o schimbare de stare, dacă o alarmă a apărut după o citire de senzor sau dacă trimiterea logurilor s-a produs după solicitarea din aplicația PWA.

Adăugarea unui eveniment în jurnal se face printr-o funcție dedicată, care primește mesajul, îl salvează în poziția curentă din buffer și actualizează indexul pentru următoarea scriere. Mesajele sunt limitate ca lungime, pentru ca fiecare intrare să ocupe un spațiu previzibil în memorie. În același timp, evenimentul este afișat și pe monitorul serial, ceea ce ajută la depanare în timpul dezvoltării. Prin această dublă utilizare, logul servește atât interfeței PWA, cât și procesului de verificare tehnică.

Transmiterea logurilor către aplicația PWA se realizează la cerere. Utilizatorul poate solicita jurnalul din interfață, iar aplicația publică o comandă MQTT pe topicul dedicat. După primirea comenzii, firmware-ul parcurge bufferul circular și construiește mesaje JSON care conțin intrările disponibile. Pentru a evita un payload prea mare, logurile sunt transmise în loturi, fiecare lot conținând un număr limitat de evenimente. Această împărțire reduce riscul depășirii dimensiunii bufferului MQTT și face comunicarea mai stabilă.

Prin mecanismul de jurnalizare, firmware-ul oferă o formă suplimentară de transparență asupra funcționării interne. Evenimentele importante nu dispar imediat după producere, ci rămân

disponibile pentru aplicația PWA și pentru testare. Această funcție închide descrierea implementării firmware-ului, deoarece leagă toate modulele analizate anterior: comunicația MQTT, senzorii, alarmele, RFID-ul, LED-ul, LCD-ul și buzzerul. În ansamblu, jurnalizarea ajută la înțelegerea comportamentului sistemului și la validarea modului în care ESP32 coordonează componentele proiectului Smart Home.

Capitolul 5. Implementarea aplicației PWA și comunicația MQTT

5.1. Structura aplicației PWA

Aplicația PWA realizată pentru sistemul Smart Home are rolul de interfață principală între utilizator și dispozitivul ESP32. Prin această aplicație, utilizatorul poate urmări valorile senzorilor, poate verifica starea alarmei, poate controla LED-ul RGB, poate gestiona cardurile RFID, poate configura anumite comportamente asociate cardurilor și poate consulta logurile sistemului. Din punct de vedere tehnic, aplicația este construită ca o interfață web responsivă, structurată într-un fișier HTML care include marcajul paginii, stilurile CSS și logica JavaScript necesară pentru comunicația MQTT și actualizarea elementelor vizuale.

Structura aplicației urmărește o organizare simplă și potrivită pentru utilizare pe dispozitive mobile. În partea superioară se află antetul, care afișează numele sistemului și starea conexiunii MQTT. Această stare este prezentată printr-un badge vizual, care indică dacă aplicația este deconectată, în curs de conectare, conectată sau în eroare. În partea inferioară este plasată o bară de navigare fixă, cu butoane pentru principalele ecrane ale aplicației. Această poziționare este potrivită pentru telefon, deoarece utilizatorul poate schimba rapid secțiunea fără să deruleze pagina.

Aplicația este împărțită în cinci ecrane principale: Dashboard, LED, RFID, Config și Log/Setări. Fiecare ecran este reprezentat printr-un container separat, iar afișarea se face prin activarea sau dezactivarea clasei CSS corespunzătoare. Această organizare permite păstrarea tuturor funcțiilor în aceeași pagină, fără încărcări repetate sau navigare între fișiere diferite. Utilizatorul are impresia unei aplicații cu mai multe pagini, însă tehnic interfața funcționează ca o aplicație web de tip single page, în care JavaScript-ul controlează ecranul vizibil.

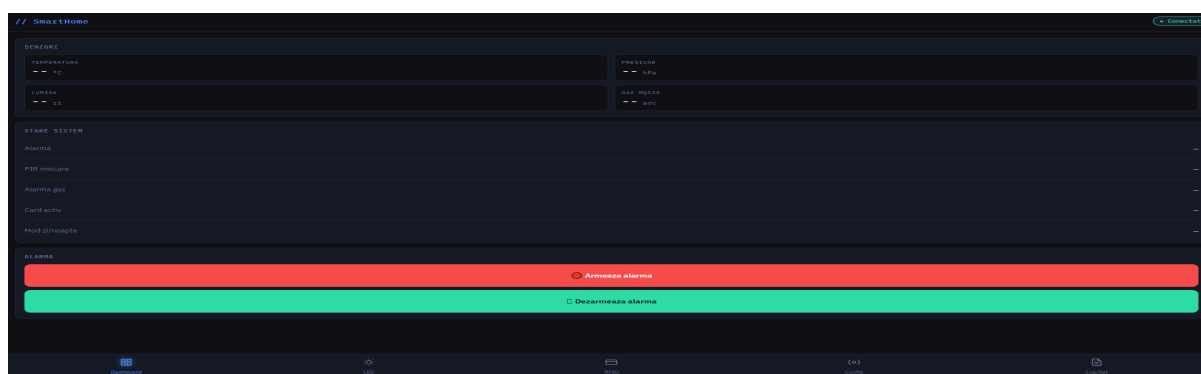


Figura 5.1. Structura ecranelor aplicației PWA

Aplicația include și mecanisme de feedback pentru utilizator. Badge-ul de conexiune indică starea legăturii MQTT, bannerul de alarmă apare atunci când sistemul semnalează un eveniment critic, iar logul local păstrează mesajele recente. În plus, aplicația poate solicita permisiune pentru notificări, astfel încât anumite evenimente importante să poată fi semnalate mai vizibil. Aceste elemente

completează interfața și ajută utilizatorul să observe atât funcționarea normală, cât și stările de avertizare.

Prin structura sa, aplicația PWA funcționează ca un panou de control pentru sistemul Smart Home. Ea nu implementează logica fizică a senzorilor sau a alarmelor, deoarece aceste responsabilități aparțin firmware-ului ESP32, dar oferă utilizatorului acces la date și comenzi printr-o interfață web clară. Această separare între dispozitivul IoT și aplicația de control susține arhitectura generală a proiectului și pregătește secțiunile următoare, în care vor fi analizate conectarea la broker, dashboard-ul, controlul LED-ului, RFID-ul, configurarea cardurilor și logurile.

5.2. Conectarea la broker prin Paho MQTT

Conectarea aplicației PWA la brokerul MQTT este realizată prin biblioteca Eclipse Paho JavaScript, încărcată în pagină printr-un script extern. Această bibliotecă permite aplicației rulate în browser să se comporte ca un client MQTT, cu posibilitatea de a se conecta la broker, de a se abona la topicuri, de a primi mesaje și de a publica payload-uri către sistemul ESP32. În arhitectura proiectului, această etapă este esențială deoarece aplicația PWA nu comunică direct cu microcontrolerul, ci folosește brokerul HiveMQ Cloud ca intermediar pentru toate mesajele de stare și comandă.

Aplicația conține o zonă dedicată setărilor de conexiune MQTT. Utilizatorul poate introduce hostul brokerului, portul WebSocket, numele de utilizator și parola. Aceste valori sunt preluate din câmpurile interfeței și sunt salvate local în browser, astfel încât aplicația să le poată reutiliza la o deschidere ulterioară. Această soluție este practică pentru prototip, deoarece reduce numărul de pași necesari la testare și permite reconectarea automată atunci când datele brokerului au fost deja completate.

Pentru fiecare sesiune, aplicația generează un identificator de client MQTT. Acest identificator pornește de la un prefix specific aplicației și este completat cu o secvență aleatoare. Rolul său este de a diferenția clientul web de alți clienți conectați la același broker, inclusiv de ESP32. Într-un sistem MQTT, identificatorul clientului este important deoarece brokerul îl folosește pentru gestionarea conexiunii. Prin generarea unui ID diferit la fiecare sesiune, aplicația reduce riscul de conflict cu o conexiune anterioară sau cu un alt client care folosește același broker.

Conexiunea propriu-zisă este creată prin instanțierea unui client Paho MQTT, folosind hostul, portul, calea WebSocket și identificatorul generat. În aplicație, calea folosită pentru comunicare este /mqtt, iar portul este citit din setările introduse în interfață. Pentru HiveMQ Cloud, portul WebSocket securizat folosit în mod obișnuit este 8884, iar aplicația activează automat opțiunea useSSL atunci când portul este 8884 sau 443. Această verificare ajută aplicația să aleagă corect între o conexiune securizată și una nesecurizată, în funcție de mediul de testare. După crearea clientului MQTT,

aplicația definește două funcții importante: una pentru pierderea conexiunii și una pentru mesajele primite.

Opțiunile de conectare includ folosirea SSL, un timp maxim de așteptare, o funcție pentru conectare reușită și o funcție pentru eșec. Dacă utilizatorul a introdus un nume de utilizator, aplicația include și parola în opțiunile de conectare. Această structură permite folosirea unui broker care solicită autentificare, cum este cazul unui cluster HiveMQ Cloud configurat cu utilizator și parolă. În cazul unei conectări reușite, aplicația marchează starea ca fiind conectată, modifică badge-ul de conexiune, adaugă un mesaj în logul local și se abonează la topicurile pe care trebuie să le urmărească.

Abonarea la topicuri se realizează imediat după conectare. Aplicația parcurge lista topicurilor definite în obiectul de configurare și trimite câte o cerere de subscribe pentru fiecare topic. Această listă include topicurile pentru senzori, alarma de gaz, alarma de mișcare, carduri RFID noi, loguri și status. Prin această operație, aplicația devine pregătită să primească mesajele publicate de ESP32. Dacă abonarea nu ar avea loc după conectare, aplicația ar putea fi conectată la broker, dar nu ar primi datele necesare pentru actualizarea interfeței.

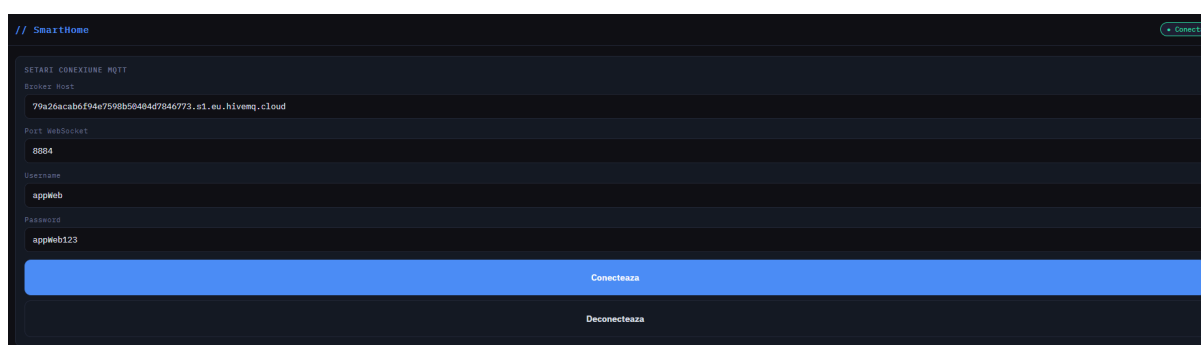


Figura 5.2. Secvența de conectare a aplicației PWA la brokerul HiveMQ

Aplicația include și funcții pentru publicarea mesajelor către broker. Pentru comenzile care folosesc payload JSON, funcția de publicare transformă obiectul JavaScript într-un șir JSON și îl trimite pe topicul primit ca argument. Această funcție este folosită pentru comenzi precum armarea alarmei, controlul LED-ului RGB, ștergerea unui card RFID sau salvarea configurației unui card. Pentru comenzile simple, aplicația include și o funcție de publicare raw, care trimite direct textul necesar, de exemplu pentru solicitarea logurilor sau pentru activarea modului de adăugare RFID.

Înainte de publicarea unui mesaj, aplicația verifică dacă există o conexiune MQTT activă. Dacă utilizatorul încearcă să trimită o comandă fără ca aplicația să fie conectată, mesajul nu este publicat, iar în logul local apare o atenționare. Această verificare previne situațiile în care utilizatorul crede că a transmis o comandă, deși clientul MQTT nu are o conexiune validă. Într-o interfață de control Smart Home, această confirmare este importantă deoarece comenzile au efect asupra unui dispozitiv fizic.

Starea conexiunii este reflectată permanent în interfață prin badge-ul din antet. Aplicația poate afișa stări precum deconectat, conectare, conectat sau eroare. Această informare vizuală ajută utilizatorul să înțeleagă rapid dacă valorile din dashboard sunt actualizate și dacă comenzile pot fi trimise către ESP32. În lipsa unui astfel de indicator, o problemă de rețea ar putea fi confundată cu o defecțiune a senzorilor sau a firmware-ului.

Prin conectarea la broker prin Paho MQTT, aplicația PWA devine un client MQTT complet în cadrul arhitecturii proiectului. Ea se abonează la mesajele publicate de ESP32, interpretează payload-urile primite, actualizează interfața și publică mesaje de comandă către microcontroler. Această legătură este baza funcțiilor descrise în secțiunile următoare, deoarece dashboard-ul, controlul LED-ului, gestiunea RFID, configurarea cardurilor și afișarea logurilor depind toate de conexiunea MQTT stabilită în această etapă.

5.3. Dashboard-ul pentru monitorizare

Dashboard-ul reprezintă ecranul principal al aplicației PWA și are rolul de a concentra informațiile esențiale despre starea sistemului Smart Home. Prin această secțiune, utilizatorul poate vedea rapid valorile transmise de ESP32 și poate înțelege dacă sistemul funcționează normal sau dacă există o stare care necesită atenție. Ecranul este împărțit în zone vizuale distincte, astfel încât datele de la senzori, starea alarmei și comenzile principale să fie accesibile fără navigare suplimentară.

Prima zonă a dashboard-ului este dedicată senzorilor. Aici sunt afișate temperatura, presiunea atmosferică, nivelul de lumină și valoarea furnizată de senzorul de gaz. Aceste date provin din mesajul MQTT publicat periodic de ESP32 pe topicul de senzori, iar aplicația le actualizează imediat după primirea payload-ului JSON. Valorile nu sunt introduse manual și nu sunt calculate în aplicație, ci reflectă datele preluate de firmware din mediul fizic. Astfel, dashboard-ul devine punctul în care informațiile citite de senzori sunt transformate în date vizibile pentru utilizator.

A doua zonă prezintă starea generală a sistemului. Aplicația afișează dacă alarma este armată sau dezarmată, dacă senzorul PIR detectează mișcare, dacă există o alarmă de gaz și dacă sistemul interpretează mediul ca fiind zi sau noapte. Pentru aceste stări, interfața folosește text și clase vizuale diferite, astfel încât utilizatorul să poată distinge rapid între o stare normală și una de avertizare. De exemplu, alarma armată sau alarma de gaz sunt evidențiate vizual, iar senzorul de gaz primește o marcare suplimentară atunci când sistemul detectează o valoare critică.

Dashboard-ul include și comenzile pentru armarea și dezarmarea alarmei. Aceste butoane permit utilizatorului să modifice direct starea sistemului prin aplicația PWA. Atunci când este apăsat un buton, aplicația publică un mesaj MQTT pe topicul de comandă al alarmei, iar ESP32 interpretează payload-ul și actualizează starea internă. Prin această funcție, dashboard-ul nu este doar un ecran de afișare, ci și un punct de control pentru una dintre cele mai importante funcții ale sistemului.

Un element vizual important este bannerul de alarmă, care apare atunci când sistemul detectează o stare critică. Acesta este afișat în partea superioară a aplicației și atrage atenția asupra evenimentelor de tip alarmă. În cazul alarmei de gaz sau al alarmei de mișcare, aplicația poate afișa un mesaj distinct și poate înregistra evenimentul în logul local. Această soluție ajută utilizatorul să observe rapid situațiile importante, fără să analizeze fiecare valoare afișată în dashboard.

Prin structura sa, dashboard-ul oferă o imagine rapidă asupra sistemului Smart Home. Utilizatorul poate verifica valorile de mediu, poate observa starea alarmei și poate trimite o comandă de armare sau dezarmare din același ecran. Această organizare susține ideea unei aplicații de control simplificate, în care informațiile tehnice sunt prezentate într-o formă ușor de interpretat.

5.4. Controlul LED RGB

Ecranul dedicat controlului LED-ului RGB permite utilizatorului să modifice direct culoarea elementului de semnalizare conectat la ESP32. În această secțiune, aplicația PWA oferă trei controale de tip slider, corespunzătoare componentelor roșu, verde și albastru. Prin modificarea acestor valori, utilizatorul poate construi o culoare personalizată, iar interfața actualizează local zona de previzualizare pentru a arăta rezultatul aproximativ înainte de trimiterea comenzii către sistem.

Controlul LED-ului este realizat prin MQTT. Atunci când utilizatorul trimite culoarea aleasă, aplicația construiește un payload JSON care conține valorile r, g și b, apoi îl publică pe topicul `smarthome/cmd/light`. Firmware-ul ESP32 primește mesajul, îl interpretează în funcția de callback și actualizează valorile PWM ale celor trei canale ale LED-ului. În acest mod, o acțiune realizată în interfața web se transformă într-o modificare vizibilă a componentei hardware.

Pe lângă alegerea manuală a culorii, aplicația include și presetări rapide. Acestea permit selectarea imediată a unor culori uzuale, fără reglarea individuală a fiecărui slider. De asemenea, LED-ul poate fi oprit printr-o comandă dedicată, care transmite firmware-ului faptul că ieșirea RGB trebuie setată la valoarea zero. Această funcție este utilă pentru testare și pentru revenirea rapidă la o stare neutră.

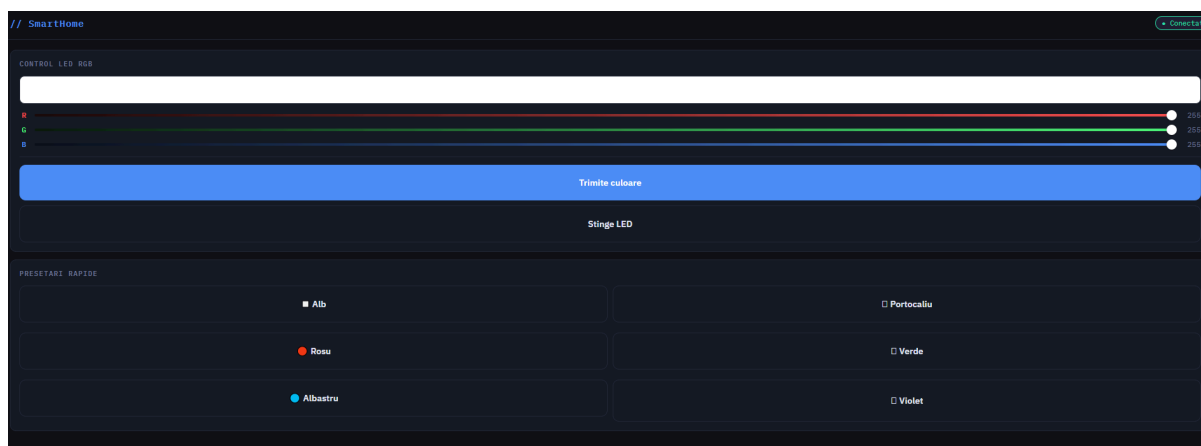


Figura 5.3. Ecranul de control al LED-ului RGB

Prin această funcție, aplicația demonstrează controlul la distanță al unui actuator fizic. LED-ul RGB nu este folosit doar pentru semnalizare automată în firmware, ci poate fi modificat și manual de utilizator, ceea ce evidențiază caracterul interactiv al sistemului Smart Home.

5.5. Gestiunea RFID

Gestiunea RFID din aplicația PWA permite utilizatorului să adauge carduri noi, să șteargă carduri existente și să folosească UID-urile detectate pentru configurații ulterioare. Această funcție completează partea de firmware descrisă anterior, deoarece aplicația nu citește direct cardurile RFID, ci trimite comenzi către ESP32 prin MQTT. Citirea efectivă rămâne responsabilitatea modulului RC522 conectat la microcontroler, iar aplicația oferă doar interfața prin care utilizatorul inițiază și controlează operațiile.

Ecranul RFID este împărțit în trei zone principale. Prima zonă permite adăugarea unui card nou. Utilizatorul apasă butonul de adăugare, iar aplicația publică o comandă pe topicul `smarthome/cmd/rfid/add`. După primirea acestei comenzi, ESP32 intră pentru o perioadă limitată în modul de așteptare, iar utilizatorul trebuie să apropie cardul de cititor. Dacă firmware-ul citește un card valid și îl salvează, aplicația primește ulterior un mesaj pe topicul `smarthome/rfid/new` și actualizează interfața.

A doua zonă permite ștergerea unui card pe baza UID-ului. Utilizatorul introduce identificatorul cardului în câmpul dedicat, iar aplicația îl transformă într-un format uniform, cu litere mari, înainte de a trimite comanda. Mesajul este publicat pe topicul `smarthome/cmd/rfid/del` și conține UID-ul cardului care trebuie eliminat. Firmware-ul caută cardul în lista memorată, îl șterge dacă există și actualizează memoria nevolatilă.

A treia zonă afișează cardurile detectate recent. Atunci când ESP32 anunță prin MQTT că un card nou a fost adăugat, aplicația introduce UID-ul în această listă și îl afișează împreună cu indexul asociat. Pentru fiecare card detectat, interfața oferă acțiuni rapide, precum completarea automată a

câmpului de ștergere sau trimiterea UID-ului către ecranul de configurare. Această funcție simplifică utilizarea aplicației, deoarece utilizatorul nu trebuie să copieze manual UID-ul.

Prin această organizare, aplicația PWA separă clar operațiile RFID: adăugarea pornește un mod temporar de așteptare pe ESP32, ștergerea trimite un UID explicit, iar lista de carduri recente ajută la refolosirea rapidă a identificatorilor. Gestiunea RFID demonstrează modul în care o interfață web poate controla o componentă hardware locală fără conectare directă la aceasta, folosind brokerul MQTT ca intermediar între utilizator și firmware.

Configurarea cardurilor RFID permite asocierea unor setări individuale pentru fiecare card memorat în sistem. În aplicația PWA, această funcție este disponibilă într-un ecran separat, unde utilizatorul poate introduce UID-ul cardului și poate stabili comportamentul pe care ESP32 trebuie să îl aplice atunci când acel card este citit. Prin această funcție, cardul RFID nu mai are doar rol de identificare, ci devine un element care poate modifica răspunsul local al sistemului.

Ecranul de configurare conține un câmp pentru UID și controale pentru alegerea culorii LED-ului RGB. Utilizatorul poate regla separat valorile pentru roșu, verde și albastru, iar aplicația actualizează o zonă de previzualizare pentru culoarea selectată. Această previzualizare este utilă deoarece permite verificarea vizuală a rezultatului înainte ca setarea să fie trimisă către ESP32. Dacă UID-ul provine din lista de carduri detectate recent, aplicația îl poate completa automat, reducând riscul de introducere greșită.

Prin această funcție, aplicația PWA oferă un nivel suplimentar de personalizare a sistemului Smart Home. Utilizatorul poate decide cum reacționează LED-ul și buzzerul pentru fiecare card, iar ESP32 aplică aceste setări în momentul citirii RFID. Configurarea cardurilor completează gestiunea RFID și arată cum aplicația poate modifica date persistente din firmware printr-un flux simplu de comenzi MQTT.

5.6. Loguri, setări MQTT și persistență locală

Ecranul Log/Setări are rolul de a grupa funcțiile auxiliare ale aplicației PWA, adică acele funcții care nu țin direct de monitorizarea senzorilor sau de controlul componentelor, dar sunt necesare pentru utilizarea stabilă a sistemului. În această secțiune, utilizatorul poate configura conexiunea MQTT, poate solicita logurile de la ESP32 și poate urmări evenimentele recente afișate în interfață. Prin această organizare, aplicația separă zona de control propriu-zis de zona de configurare și diagnostic.

Setările MQTT includ hostul brokerului, portul, numele de utilizator și parola. Aceste date sunt introduse în câmpurile aplicației și sunt salvate local în browser, folosind mecanismul localStorage. Astfel, la o deschidere ulterioară a aplicației, valorile pot fi reîncărcate automat în interfață, fără ca utilizatorul să le introducă din nou. Această persistență locală este utilă în etapa de

testare, deoarece conexiunea la broker trebuie refăcută de mai multe ori, iar completarea repetată a datelor ar îngreuna utilizarea aplicației.

Zona de loguri afișează atât evenimente locale generate de aplicație, cât și evenimente primite de la ESP32. De exemplu, aplicația poate adăuga în log mesaje despre conectarea la broker, pierderea conexiunii, trimiterea unei comenzi de armare, controlul LED-ului sau solicitarea jurnalului firmware-ului. În același timp, atunci când ESP32 trimite logurile sale prin MQTT, aplicația le inserează în aceeași zonă, marcându-le ca evenimente provenite de la dispozitiv.

Solicitarea logurilor se face prin publicarea unui mesaj pe topicul `smarthome/cmd/log`. După primirea comenzii, ESP32 trimite intrările memorate în bufferul său circular, iar aplicația le afișează în ordine în interfață. Această funcție este utilă pentru verificarea comportamentului sistemului, deoarece utilizatorul poate observa ce evenimente au fost înregistrate de firmware, chiar dacă nu urmărea dashboard-ul în momentul producerii lor.

Aplicația păstrează și un jurnal local, limitat la un număr rezonabil de intrări, pentru a evita aglomerarea interfeței. Evenimentele noi sunt adăugate în partea superioară, iar cele mai vechi pot fi eliminate atunci când lista devine prea mare. Utilizatorul are și posibilitatea de a șterge logul local, ceea ce este util în timpul testelor repetate. Această funcție nu afectează direct memoria ESP32, ci doar afișarea din aplicația PWA.

Capitolul 6. Concluzii

Testarea sistemului Smart Home a urmărit verificarea funcționării integrate a celor două componente principale ale proiectului: firmware-ul rulat pe ESP32 și aplicația PWA conectată la brokerul MQTT. Scopul testării nu a fost obținerea unei certificări tehnice a sistemului, ci confirmarea faptului că prototipul îndeplinește cerințele stabilite anterior și că poate realiza fluxurile esențiale de monitorizare, control, alarmare, identificare RFID și jurnalizare. Testele au fost realizate gradual, pornind de la verificarea componentelor hardware individuale și continuând cu validarea comunicației MQTT și a interacțiunii dintre aplicație și microcontroler.

Prima etapă de testare a vizat pornirea sistemului și inițializarea componentelor. După alimentarea plăcii ESP32, au fost urmărite mesajele afișate pe LCD și în monitorul serial, pentru a confirma că firmware-ul intră în etapa de inițializare, configurează pini, pornește interfețele I2C și SPI, încarcă datele RFID din memoria nevolatilă și încearcă stabilirea conexiunii WiFi. Rezultatul acestei verificări a fost pozitiv, sistemul reușind să pornească într-o stare stabilă și să afișeze mesaje locale care indică progresul inițializării.

A doua etapă a constat în verificarea conectării MQTT. Aplicația PWA a fost configurată cu datele brokerului, iar ESP32 a fost conectat la aceeași infrastructură de comunicație prin topicurile definite în proiect. S-a urmărit dacă aplicația își schimbă starea din deconectat în conectat, dacă ESP32 publică mesajul de status și dacă dashboard-ul începe să primească date de la senzori. Rezultatul a confirmat că brokerul MQTT permite schimbul de mesaje între aplicație și firmware, iar interfața PWA poate funcționa ca panou de control pentru sistem.

Pentru validarea senzorilor, au fost observate valorile afișate în dashboard pentru temperatură, presiune, lumină, gaz și starea PIR. Datele publicate periodic de ESP32 au fost recepționate corect de aplicație, iar valorile s-au actualizat în interfață fără intervenție manuală. Senzorul de lumină a influențat starea zi/noapte, senzorul de gaz a modificat valoarea afișată în dashboard, iar senzorul PIR a indicat schimbarea stării atunci când a fost detectată mișcare. Această etapă a validat traseul datelor de la senzori, prin firmware și MQTT, până la afișarea în aplicația PWA.

Testarea funcției de alarmă a inclus armarea și dezarmarea sistemului din aplicație, simularea unei valori ridicate pentru gaz și verificarea detecției de mișcare prin senzorul PIR. La armare, firmware-ul a schimbat starea internă, iar aplicația a afișat corespunzător noua stare. La dezarmare, stările de alarmă asociate mișcării au fost resetate, iar buzzerul a fost oprit. În cazul alarmei de gaz, sistemul a semnalizat evenimentul prin MQTT, prin feedback local și prin actualizarea interfeței. În cazul mișcării, sistemul a trecut mai întâi prin perioada silențioasă, apoi a activat alarma dacă dezarmarea nu a fost realizată. Aceste rezultate arată că logica de alarmă funcționează coerent și respectă prioritățile stabilite în firmware.

Controlul LED-ului RGB a fost verificat prin modificarea valorilor RGB din aplicația PWA și prin folosirea presetărilor rapide. Comenzile trimise prin MQTT au fost recepționate de ESP32, iar LED-ul a răspuns prin schimbarea culorii. De asemenea, s-a observat că stările de alarmă au prioritate față de controlul manual, ceea ce confirmă faptul că firmware-ul nu tratează LED-ul doar ca actuator decorativ, ci ca element de semnalizare integrat în logica sistemului. LCD-ul a fost verificat prin afișarea mesajelor de pornire, a mesajelor temporare și a paginilor de stare, iar buzzerul a fost verificat atât pentru feedback scurt, cât și pentru semnalizarea alarmelor.

Gestiunea RFID a fost testată prin adăugarea unui card nou, citirea cardului, ștergerea lui și configurarea unor setări asociate. După comanda de adăugare trimisă din aplicație, ESP32 a intrat în modul de așteptare, iar apropierea cardului de cititor a produs memorarea UID-ului. Cardul adăugat a apărut în interfață, iar UID-ul a putut fi folosit ulterior pentru ștergere sau configurare. După repornirea sistemului, datele RFID salvate au rămas disponibile, ceea ce confirmă funcționarea stocării persistente prin memoria nevolatilă a ESP32.

Funcția de log a fost validată prin generarea unor evenimente succesive, precum conectarea MQTT, armarea alarmei, controlul LED-ului, citirea RFID și solicitarea jurnalului din aplicație. Logurile locale ale aplicației au afișat comenzile și mesajele importante, iar logurile trimise de ESP32 au oferit o confirmare suplimentară a evenimentelor produse în firmware. Această funcție s-a dovedit utilă pentru observarea comportamentului sistemului și pentru verificarea etapelor de testare fără a depinde exclusiv de monitorul serial.

Rezultatele obținute arată că prototipul îndeplinește obiectivele funcționale propuse. Sistemul poate citi senzori, poate transmite date prin MQTT, poate afișa valorile într-o aplicație PWA, poate primi comenzi de control, poate gestiona alarme, poate folosi RFID pentru identificare și poate păstra loguri relevante. Limitările observate țin în principal de caracterul demonstrativ al proiectului: dependența de brokerul MQTT, lipsa unui sistem avansat de autentificare în aplicație, sensibilitatea senzorilor la condițiile de montaj și caracterul orientativ al pragurilor folosite. Cu toate acestea, pentru scopul lucrării, validarea confirmă că arhitectura aleasă este funcțională și că integrarea dintre ESP32, MQTT și aplicația PWA a fost realizată cu succes.

Lucrarea de față a avut ca scop proiectarea și implementarea unui sistem Smart Home funcțional, bazat pe placa ESP32, o aplicație PWA și comunicație MQTT printr-un broker cloud. Proiectul a urmărit realizarea unui prototip capabil să monitorizeze parametri ai mediului, să transmită date către o interfață web, să primească de la utilizator comenzi de control și să reacționeze local în situații de alarmă. Prin integrarea componentelor hardware cu o aplicație accesibilă din browser, sistemul demonstrează modul în care un microcontroler conectat la internet poate deveni nucleul unei soluții IoT aplicate domeniului Smart Home.

Tema aleasă răspunde unei direcții actuale de dezvoltare tehnologică, în care locuințele devin tot mai conectate, iar utilizatorii au nevoie de sisteme capabile să ofere control, automatizare și informații în timp real. În acest context, lucrarea nu se limitează la prezentarea teoretică a conceptului de Smart Home, ci propune o implementare practică, în care senzorii, actuatorii, comunicația MQTT și interfața PWA sunt conectate într-un sistem coerent. Rezultatul este un prototip care poate fi observat, comandat și testat, oferind o bază concretă pentru extinderi ulterioare.

Din punct de vedere hardware, sistemul integrează mai multe componente cu roluri diferite. ESP32 reprezintă unitatea centrală de control, iar senzorii conectați permit monitorizarea unor valori relevante pentru mediul interior. BMP180 este folosit pentru temperatură și presiune, TEMT6000 pentru nivelul de lumină, MQ135 pentru detectarea variațiilor asociate calității aerului sau prezenței gazelor, iar senzorul PIR pentru detectarea mișcării. Pe lângă senzori, prototipul include LED RGB, buzzer, afișaj LCD și modul RFID RC522, ceea ce permite atât semnalizare locală, cât și interacțiune fizică prin carduri.

O contribuție importantă a lucrării constă în integrarea acestor componente într-un firmware unitar. Codul pentru ESP32 nu tratează fiecare modul ca element izolat, ci coordonează citirea senzorilor, stările de alarmă, comenzile MQTT, LED-ul, LCD-ul, buzzerul, RFID-ul și logurile într-o buclă principală de execuție. Această organizare permite sistemului să reacționeze periodic la valorile citite, să primească mesaje din aplicație și să ofere feedback local fără ca o funcție să blocheze complet restul programului. Prin folosirea temporizărilor și a unei structuri logice clare, firmware-ul devine capabil să gestioneze simultan mai multe responsabilități.

Comunicația MQTT bidirecțională reprezintă un alt element esențial al proiectului. ESP32 publică datele senzorilor, evenimentele de alarmă, statusul, mesajele RFID și logurile, iar aplicația PWA publică la rândul ei comenzi pentru alarmă, LED, LCD, RFID și configurarea cardurilor. Brokerul MQTT funcționează ca intermediar între cele două componente, astfel încât aplicația nu trebuie să comunice direct cu microcontrolerul. Această soluție reflectă o arhitectură specifică sistemelor IoT, în care dispozitivele și interfețele software schimbă mesaje pe topicuri bine definite.

Aplicația PWA constituie o altă contribuție personală importantă. Aceasta oferă un dashboard pentru monitorizarea senzorilor, o zonă pentru controlul LED-ului RGB, o interfață pentru gestiunea cardurilor RFID, o secțiune pentru configurarea comportamentului fiecărui card și o zonă pentru loguri și setări MQTT. Prin această aplicație, utilizatorul poate urmări în timp real valorile publicate de ESP32 și poate trimite comenzi către sistem fără a instala o aplicație nativă. Interfața este organizată astfel încât funcțiile principale să fie accesibile rapid de pe telefon, iar starea conexiunii MQTT să fie vizibilă permanent.

Controlul LED-ului RGB demonstrează transmiterea unei comenzi din aplicație către un actuator fizic. Utilizatorul poate regla valorile pentru roșu, verde și albastru, iar ESP32 aplică aceste

valori prin semnale PWM. Totodată, firmware-ul stabilește priorități, astfel încât stările de alarmă să poată suprascrie controlul manual atunci când situația o impune. Această logică arată că LED-ul nu este doar un element decorativ, ci și un indicator de stare integrat în comportamentul general al sistemului.

Modulul RFID adaugă proiectului o dimensiune suplimentară de interacțiune. Cardurile pot fi adăugate, șterse și configurate prin aplicația PWA, iar datele asociate sunt salvate în memoria nevolatilă a ESP32. Pentru fiecare card se pot stabili culoarea LED-ului, condițiile de aprindere în funcție de zi sau noapte și activarea buzzerului la citire. Această funcționalitate transformă RFID-ul dintr-un simplu mecanism de identificare într-un element personalizabil, integrat în logica sistemului Smart Home.

Sistemul include și o logică de alarmă pentru gaz și mișcare. Alarma de gaz este declanșată pe baza valorii citite de la MQ135, folosind praguri pentru activare și revenire. Alarma de mișcare este dependentă de starea de armare a sistemului și include o perioadă silențioasă înainte de activarea completă. Aceste funcții au fost implementate astfel încât evenimentele importante să fie semnalizate local prin LED, LCD și buzzer, dar și transmise către aplicația PWA prin MQTT. În acest mod, utilizatorul poate observa atât local, cât și la distanță, apariția unei stări critice.

O altă contribuție este mecanismul de jurnalizare. Firmware-ul păstrează un log circular al evenimentelor importante, iar aplicația are propriul log local pentru comenzile și mesajele primite. Prin solicitarea logurilor prin MQTT, utilizatorul poate verifica ce evenimente au avut loc în sistem, precum conectarea brokerului, armarea alarmei, citirea RFID, declanșarea alarmelor sau trimiterea unor comenzi. Această funcție este utilă pentru testare, dar și pentru înțelegerea comportamentului prototipului în timpul utilizării.

Rezultatele obținute confirmă funcționarea principalelor componente ale sistemului. ESP32 poate citi senzorii conectați și poate publica valorile către brokerul MQTT. Aplicația PWA poate primi aceste valori și le poate afișa în dashboard. Comenzile transmise din aplicație pot modifica starea alarmei, culoarea LED-ului, lista cardurilor RFID și configurațiile asociate acestora. Alarma de gaz și alarma de mișcare sunt tratate de firmware și semnalizate prin mai multe canale. Logurile pot fi generate, trimise și afișate. În ansamblu, sistemul realizează traseul complet al informației: de la mediu către senzori, de la senzori către ESP32, de la ESP32 către broker și de la broker către aplicația PWA.

Testarea practică a arătat că arhitectura propusă este funcțională și potrivită pentru un prototip Smart Home. Sistemul a putut fi pornit, conectat la rețea, conectat la broker, monitorizat din aplicație și comandat prin topicuri MQTT. De asemenea, s-a verificat faptul că anumite funcții pot continua local chiar dacă aplicația nu este urmărită permanent, cum ar fi tratarea senzorilor, semnalizarea pe LCD, reacția buzzerului și controlul LED-ului în situații de alarmă. Acest aspect este important,

deoarece un sistem Smart Home trebuie să aibă un anumit nivel de autonomie locală, nu doar o dependență completă de interfața utilizatorului.

Totuși, proiectul rămâne un prototip și are limitări firești. Pragurile pentru senzorul MQ135 sunt orientative și ar trebui calibrate mai riguros pentru utilizare reală. Aplicația PWA salvează setările MQTT local în browser, ceea ce este practic pentru testare, dar nu reprezintă o soluție avansată de securitate. Sistemul nu include un backend propriu, o bază de date pentru istoric sau un mecanism complex de autentificare. De asemenea, aplicația nu oferă încă grafice pentru evoluția senzorilor și nu implementează complet toate avantajele unei PWA, cum ar fi funcționarea offline printr-un service worker configurat complet.

Direcțiile viitoare de dezvoltare pot transforma prototipul într-o soluție mai matură. O primă îmbunătățire ar fi realizarea unui backend propriu, care să preia datele de la broker, să le valideze și să le ofere aplicației printr-o interfață controlată. În legătură cu aceasta, ar putea fi introdusă o bază de date pentru păstrarea istoricului valorilor citite de senzori, a alarmelor, a cardurilor utilizate și a comenzilor trimise. O astfel de extindere ar permite analiza comportamentului sistemului în timp și ar crea baza pentru rapoarte sau alerte mai avansate.

O altă direcție importantă este îmbunătățirea securității. Aplicația ar putea include autentificare mai sigură, conturi de utilizator, roluri și permisiuni diferențiate. De exemplu, un utilizator ar putea avea acces doar la monitorizare, în timp ce un administrator ar putea modifica alarme, șterge carduri RFID sau schimba setările MQTT. În același timp, datele sensibile ar trebui gestionate într-un mod mai sigur decât simpla salvare locală în browser.

Interfața PWA poate fi extinsă prin grafice pentru senzori, astfel încât utilizatorul să poată urmări evoluția temperaturii, presiunii, luminii și valorii MQ135 într-un interval de timp. De asemenea, aplicația ar putea include un service worker complet, care să permită instalarea mai stabilă, încărcarea rapidă și anumite funcții offline. Notificările ar putea fi transformate în notificări push reale, transmise printr-un server, astfel încât utilizatorul să fie avertizat chiar și atunci când aplicația nu este deschisă în browser.

Din punct de vedere hardware, sistemul ar putea fi îmbunătățit prin realizarea unei carcase imprimate 3D, care să fixeze componentele într-o formă sigură și estetică. O carcasă bine proiectată ar proteja placa ESP32, senzorii, afișajul și modulul RFID, oferind în același timp acces la zonele care trebuie expuse mediului, precum senzorul de gaz, senzorul de lumină sau cititorul RFID. În plus, senzorul MQ135 ar trebui calibrat mai precis, prin teste repetate și prin corelarea valorilor citite cu condițiile reale de mediu.

O posibilă extindere software ar fi integrarea cu platforme existente de automatizare, precum Home Assistant. Printr-o astfel de integrare, sistemul ar putea comunica mai ușor cu alte dispozitive inteligente, iar utilizatorul ar putea include prototipul într-un ecosistem Smart Home mai larg. Această

direcție ar transforma proiectul dintr-un sistem izolat într-un nod conectabil la o platformă de automatizare deja consacrată.

În concluzie, lucrarea demonstrează că un sistem Smart Home funcțional poate fi construit cu o arhitectură relativ accesibilă, folosind ESP32, senzori, RFID, MQTT și o aplicație PWA. Proiectul integrează componente hardware și software într-un ansamblu coerent, capabil să monitorizeze, să controleze, să semnalizeze și să păstreze evenimente relevante. Rezultatele obținute confirmă atingerea obiectivelor propuse, iar limitările identificate oferă direcții clare pentru dezvoltări viitoare. Prin caracterul său aplicativ, proiectul evidențiază potențialul tehnologiilor IoT în realizarea unor soluții Smart Home flexibile, extensibile și adaptabile nevoilor utilizatorului.

Bibliografie

- [B1] ITU-T, Recommendation ITU-T Y.4000/Y.2060, Overview of the Internet of Things, International Telecommunication Union, iunie 2012.
- [B2] Stolojescu-Crișan, C., Crisan, C., Butunoi, B.-P., „An IoT-Based Smart Home Automation System”, *Sensors*, vol. 21, nr. 11, 2021, articol 3784, DOI: 10.3390/s21113784.
- [B3] Haney, J. M., Acar, Y., Li, A., Haney, F., „Smart Home Users’ Security and Privacy Perceptions and Actions Differ By Device Category: Results from a U.S. Survey”, în Moallem, A. (ed.), *HCI for Cybersecurity, Privacy and Trust, HCII 2025, Lecture Notes in Computer Science*, vol. 15815, Springer, Cham, 2025, pp. 188–207, DOI: 10.1007/978-3-031-92840-6_11.
- [B4] Espressif Systems, ESP32 Series Datasheet, Version 5.24, Espressif Systems, document tehnic.
- [B5] Espressif Systems, ESP32 Technical Reference Manual, Version 5.7, Espressif Systems, document tehnic.
- [B6] OASIS, MQTT Version 3.1.1 Plus Errata 01, OASIS Standard Incorporating Approved Errata 01, OASIS Open, 10 decembrie 2015.
- [B7] Fagan, M., Megas, K., Scarfone, K., Smith, M., IoT Device Cybersecurity Capability Core Baseline, NIST Interagency/Internal Report NISTIR 8259A, National Institute of Standards and Technology, 2020, DOI: 10.6028/NIST.IR.8259A.
- [B8] Bosch Sensortec GmbH, BMP180 Digital Pressure Sensor, Data Sheet, document BST-BMP180-DS000-09, Revision 2.5, Bosch Sensortec GmbH, aprilie 2013.
- [B9] Vishay Semiconductors, TEMT6000X01 Ambient Light Sensor, Datasheet, document nr. 81579, Vishay Semiconductors.
- [B10] Zhengzhou Winsen Electronics Technology Co., Ltd., MQ135 Semiconductor Sensor for Air Quality, Manual Version 1.4, Zhengzhou Winsen Electronics Technology Co., Ltd., 10 martie 2015.
- [B11] Silvan Chip Electronics Tech. Co., Ltd., BISS0001 PIR Controller, Datasheet, Silvan Chip Electronics Tech. Co., Ltd.
- [B12] Hitachi, HD44780U (LCD-II), Dot Matrix Liquid Crystal Display Controller/Driver, ADE-207-272(Z), Revision 0.0, Hitachi, septembrie 1999.
- [B13] NXP Semiconductors, MFRC522, Standard Performance MIFARE and NTAG Frontend, Product Data Sheet, NXP Semiconductors, 27 aprilie 2016.

Referințe Web

[RW1] National Institute of Standards and Technology, „Internet of Things - Glossary”, NIST Computer Security Resource Center, disponibil la:

https://csrc.nist.gov/glossary/term/internet_of_things.

[RW2] Espressif Systems, „LED Control (LEDC) - ESP32”, ESP-IDF Programming Guide, disponibil la: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/ledc.html>.

[RW3] HiveMQ, „Getting Started with HiveMQ Cloud”, HiveMQ Cloud Documentation, disponibil la: <https://docs.hivemq.com/hivemq-cloud/quick-start-guide.html>.

[RW4] Eclipse Foundation, „Eclipse Paho JavaScript Client”, Eclipse Paho Documentation, disponibil la: <https://eclipse.dev/paho/clients/js/>.

[RW5] MDN Web Docs, Mozilla Contributors, „Progressive web apps”, Mozilla Developer Network, disponibil la: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps.

[RW6] MDN Web Docs, Mozilla Contributors, „Making PWAs installable”, Mozilla Developer Network, disponibil la: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Making_PWAs_installable.

[RW7] MDN Web Docs, Mozilla Contributors, „Service Worker API”, Mozilla Developer Network, disponibil la: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API.

[RW8] MDN Web Docs, Mozilla Contributors, „Notifications API”, Mozilla Developer Network, disponibil la: https://developer.mozilla.org/en-US/docs/Web/API/Notifications_API.

[RW9] MDN Web Docs, Mozilla Contributors, „WebSocket API (WebSockets)”, Mozilla Developer Network, disponibil la: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API.

[RW10] OWASP Foundation, „OWASP Internet of Things”, OWASP Project, disponibil la: <https://owasp.org/www-project-internet-of-things/>.

[RW11] HiveMQ, „Securing MQTT Systems - MQTT Security Fundamentals”, HiveMQ Blog, disponibil la: <https://www.hivemq.com/blog/mqtt-security-fundamentals-securing-mqtt-systems/>.

[RW12] Arduino, „LiquidCrystal”, Arduino Documentation, disponibil la: <https://docs.arduino.cc/libraries/liquidcrystal>.

[RW13] Arduino, „MFRC522”, Arduino Documentation, disponibil la: <https://docs.arduino.cc/libraries/mfrc522/>.

[RW14] Arduino, „PubSubClient”, Arduino Documentation, disponibil la: <https://docs.arduino.cc/libraries/pubsubclient/>.

[RW15] Adafruit, „Adafruit BMP085 Library”, GitHub Repository, disponibil la:

<https://github.com/adafruit/Adafruit-BMP085-Library>.

[RW16] Blanchon, B., „Documentation, ArduinoJson 6”, ArduinoJson Documentation, disponibil la:

<https://arduinojson.org/v6/doc/>.

A. CODUL SURSĂ

Codul sursa complet al firmware-ului ESP32 si al aplicației web este disponibil în repository-ul public GitHub al proiectului. Fișierele principale: SmartHome_ESP32.ino și smarthome.html.

Link GitHub: https://github.com/bacanoiuancaw2n-lgtm/Proiect_Licenta

B. SITE-UL WEB AL PROIECTULUI

Link GitHub: https://github.com/bacanoiuancaw2n-lgtm/Proiect_Licenta

C. CD / DVD / STICK USB

Link GitHub: https://github.com/bacanoiuancaw2n-lgtm/Proiect_Licenta